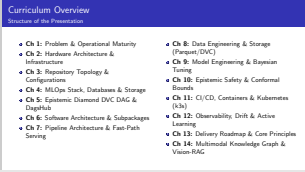
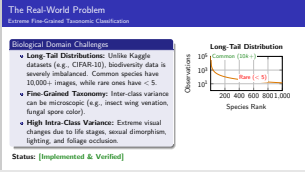


1. **Script:** Welcome everyone. Today we are walking through the TaxonVision-MLOps Strategy Report. This presentation is designed specifically for data scientists and engineers transitioning from introductory machine learning to production-grade MLOps. We will explore how real-world biodiversity classification forces us to rethink everything from storage and GPU time-slicing to mathematical safety bounds and continuous deployment.
2. **Q:** *What is the primary objective of this presentation?*
   A: To provide a comprehensive, end-to-end breakdown of the TaxonVision MLOps architecture, explaining why toy machine learning workflows fail at scale and how enterprise-grade systems guarantee reproducibility, latency, and epistemic safety.

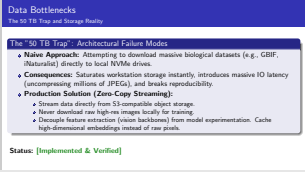


1. **Script:** Here is our roadmap for the presentation. We progress systematically from real-world biodiversity problems and bare-metal hardware through repository architecture, multi-tier storage, our Epistemic Diamond DVC DAG, modular software architecture, data engineering, model optimization, mathematical uncertainty guarantees, CI/CD, and active learning, culminating in our cutting-edge Multimodal Knowledge Graph and Vision-RAG architecture.
2. **Q:** *How is this curriculum organized?*
   A: It is structured into 14 modular chapters, moving from bare-metal foundations, repository topology, and reproducible training through mathematical calibration and deployment, concluding with deep multimodal reasoning and Vision-RAG.

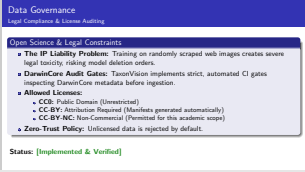
1. **Script:** Welcome to Chapter 1. We begin by examining the core real-world challenges in biodiversity classification, why toy machine learning workflows break down, and how we establish Level 4 operational maturity.
2. **Q:** *What is the primary focus of this chapter?*
   A: To analyze the extreme long-tail distribution and morphological complexity of taxonomic data, demonstrating why naive ML approaches fail and establishing the Level 4 MLOps maturity standard.



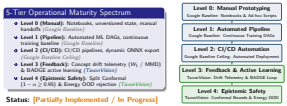
1. **Script:** When most students build computer vision models, they work with clean datasets like Dogs vs Cats or ImageNet. In real biodiversity monitoring, we deal with extreme long-tail distributions: a few common birds have 50,000 photos, while thousands of rare insects have two photos. Worse, two distinct species may differ only by microscopic vein patterns, while two photos of the exact same plant look completely different across seasons. We need a system that handles extreme fine-grained taxonomy without hallucinating confidence.
2. **Q:** *Why can't we simply collect more images for the rare species?*
   A: Biological rarity is governed by nature, not data collection budgets. An endangered lichen or beetle may only be sighted once every few years. The long-tail distribution is an immutable domain reality that our learning algorithms must solve mathematically.



1. **Script:** Here is the most common mistake made by data scientists transitioning from bootcamps to industry: they run a script to download the entire dataset onto their local SSD. At 50 terabytes, your drive fills up in minutes, your network bandwidth is choked for days, and your coworker cannot reproduce your experiment without repeating the download. We call this the '50 TB Trap'. In TaxonVision, we enforce zero-copy streaming directly from object storage and cache immutable high-dimensional embeddings instead of raw pixels.
2. **Q:** *Does streaming images directly from S3 slow down model training?*
   A: If you train an end-to-end model on raw pixels from S3, network bandwidth becomes your bottleneck. That is why our architecture decouples feature extraction from classifier training: backbones process images in batch streams once, and classifiers train in seconds from pre-cached numerical embeddings.

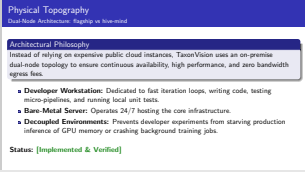


1. **Script:** Many engineers believe data pipelines are purely technical, but legal compliance is a hard engineering requirement. If your training set accidentally absorbs proprietary or copyrighted images, your entire model weights can become legally toxic and subject to deletion orders. We built an automated license audit script directly into our CI gate that inspects DarwinCore metadata and guarantees that only CC0, CC-BY, and CC-BY-NC images ever enter our pipelines.
2. **Q:** *What happens if an ingested image has no license field in its metadata?*
   A: The audit script rejects it by default under a strict zero-trust policy. Unlicensed data is treated as copyrighted until explicit open licensing is proven.

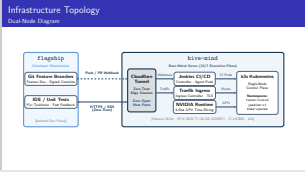


1. **Script:** To evaluate where TaxonVision sits, we adapt Google's MLOps Maturity Model. Google's standard whitepaper defines Levels 0 through 2—moving from manual Jupyter notebooks up to automated CI/CD pipelines. However, biological computer vision introduces severe risks of silent overconfidence. We explicitly extend Google's model to Level 3 (Active Learning feedback loops and drift telemetry) and Level 4 (Epistemic Safety via Split Conformal bounds and Energy-Based OOD rejection). TaxonVision operates at Level 3/4 maturity on bare-metal hardware.
2. **Q:** *Why is Level 3/4 necessary beyond Google's Level 2 baseline?*
   A: Google's Level 2 guarantees code and model deployment automation, but not statistical output safety. Levels 3 and 4 add distribution-free coverage guarantees and active learning triage to prevent uncalibrated predictions in production.

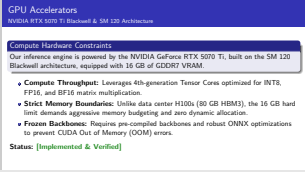
1. **Script:** In Chapter 2, we dive into the physical bare-metal foundation: our dual-node cluster topology, RTX 5070 Ti GPU time-slicing invariants, and zero-trust WireGuard networking.
2. **Q:** *Why does bare-metal infrastructure matter in MLOps?*
   A: Real-world deployment requires strict resource boundaries, predictable VRAM allocation, and thermal stability that cannot be taken for granted in production environments.



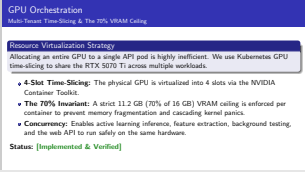
1. **Script:** In cloud-centric courses, instructors often assume you have an unlimited AWS budget. In reality, renting multiple A100 GPUs 24/7 costs thousands of dollars every month. We designed an on-premise dual-node topology: 'flagship' is our local development workstation where engineers develop with fast iteration loops, and 'hive-mind' is a 24/7 bare-metal server that hosts our k3s Kubernetes cluster, Jenkins automation, and production workloads.
2. **Q:** *Why not run everything on a single powerful machine?*
   A: Decoupling development from production prevents developer experiments from starving production inference of GPU memory or crashing background pipeline jobs.



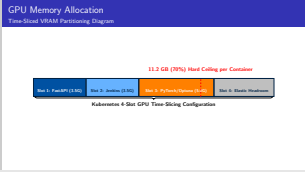
1. **Script:** This diagram visualizes our physical separation. The flagship node is strictly for local dev. Once code is pushed, Jenkins running inside the hive-mind k3s cluster takes over. The Cloudflare Tunnel securely routes external requests to our Traefik ingress without exposing any ports.
2. **Q:** *Is the network traffic between these nodes encrypted?*
   A: Yes, internal traffic can be routed over local LAN, while external web hooks and operator access run through the Cloudflare Zero Trust encrypted tunnel.



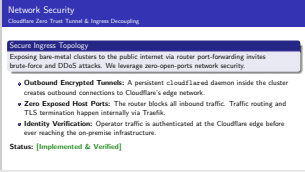
1. **Script:** Our bare-metal compute engine is powered by an NVIDIA RTX 5070 Ti based on the cutting-edge Blackwell SM 120 architecture. While it delivers extraordinary compute throughput with 4th-generation Tensor Cores and GDDR7 bandwidth, it has exactly 16 gigabytes of VRAM. If your model allocates 16.1 gigabytes, your job crashes instantly with a CUDA Out of Memory error. This physical constraint dictates our entire architectural design.
2. **Q:** *Why not use distributed multi-GPU training with cloud instances?*
   A: A single modern GPU on bare metal is more than capable of handling enterprise-scale classification if you engineer your pipelines correctly. Bounding ourselves to 16 GB forces disciplined, efficient engineering rather than brute-force cloud spending.



1. **Script:** In enterprise Kubernetes, allocating an entire physical GPU to a single API pod is massively wasteful because inference only uses the GPU intermittently. We implemented 4-slot GPU time-slicing, allowing our cluster to share the RTX 5070 Ti across multiple microservices. To ensure that one runaway job doesn't crash the entire GPU, we enforce an ironclad 70 percent VRAM cap—11.2 gigabytes—giving us a safety margin against memory fragmentation.
2. **Q:** *Does GPU time-slicing isolate memory between containers?*
   A: Time-slicing shares compute cycles in temporal slices, but does not provide hardware-enforced memory partitioning like MIG (Multi-Instance GPU). That is exactly why our 70% software cap and zero-allocation inference invariants are critical to prevent crashes.

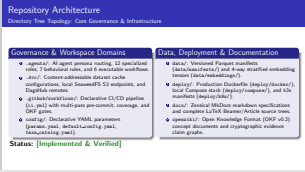


1. **Script:** This diagram shows the 16 GB VRAM physical capacity broken down into our 4 time-sliced slots. Slot 1 runs our FastAPI serving at up to 3.5 GB. Slot 2 is our Jenkins CI agent. Slot 3 is gated for PyTorch training at 5 GB. We deliberately leave Slot 4 as elastic headroom. The red dashed line illustrates our 70% container software ceiling, ensuring no single pod can monopolize the card and cause OOM cascading failures.
2. **Q:** *How is the 5.0 GB limit for PyTorch enforced?*
   A: We use `torch.cuda.set_per_process_memory_fraction(0.3125)` to explicitly restrict PyTorch to exactly 5.0 GB of the 16.0 GB total, guaranteeing it respects its designated slot bounds.

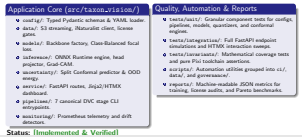


1. **Script:** How do we safely access our on-premise cluster from anywhere in the world without exposing it to internet hackers? We do not open any ports on our router. Instead, a lightweight daemon called cloudflared establishes an outbound encrypted tunnel to Cloudflare's global edge network. When an operator connects, Cloudflare authenticates their identity and proxies the traffic securely into our internal Kubernetes ingress controller.
2. **Q:** *What happens if the local internet connection resets its dynamic IP address?*
   A: The tunnel daemon maintains persistent outbound connections. If the local public IP changes, the daemon automatically reconnects without any downtime or manual DNS reconfiguration.

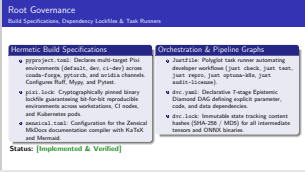
1. **Script:** In Chapter 3, we dive into the physical repository architecture: examining how our monorepo is cleanly organized into specialized domains, how parameters are hierarchically resolved, and how automated governance guarantees software quality.
2. **Q:** *Why maintain a monorepo for an MLOps pipeline?*
   A: A monorepo eliminates distributed version drift between data schemas, PyTorch models, ONNX engines, Kubernetes manifests, and documentation, binding code, parameters, and tensor hashes into a single cryptographic commit lineage.



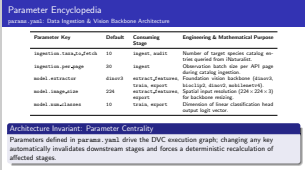
1. **Script:** Here we inspect the core system directories. Notice how operational concerns are cleanly segregated: agentic instructions in dot-agents, storage configurations in dot-dvc, deployment manifests in deploy, and our living knowledge base in openwiki.
2. **Q:** *What is the function of the .agents directory?*
   A: It provides deterministic operational constraints, preventing AI assistants from executing bare pip or non-Pixi commands and routing tasks to narrow domain roles.



1. **Script:** On the execution side, taxon-vision is modularized into high-cohesion, low-coupling subpackages under src. Every single function maintains a cognitive complexity score strictly below 15, verified by complexipy, while our comprehensive test suite guarantees coverage above 80 percent.
2. **Q:** *How is cognitive complexity enforced across the codebase?*
   A: We run complexipy across the entire src directory in pre-commit hooks and CI pipelines, rejecting any function exceeding a score of 15.



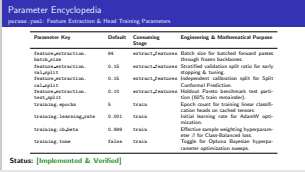
1. **Script:** The root of the repository houses our build specifications. Notice that we completely ban virtual environments and raw pip. Instead, Pixi and pyproject.toml define hermetic environments, while Justfile provides standardized recipes across all environments.
2. **Q:** *Why does the project use Justfile instead of a traditional Makefile?*
   A: Justfile provides cross-platform recipe execution, strict error handling, built-in parameterization, and seamless integration with Pixi environments without confusing tab/space syntax traps.


Parameter Encyclopedia
params.yaml: Data Ingestion & Vision Backbone Architecture

Parameter Key	Default	Consuming Stage	Engineering & Mathematical Purpose
ingestion.taxa_to_fetch	10	ingest_audit	Number of target species catalog entries queried from iNaturalist.
ingestion.per_page	30	ingest	Observation batch size per API page during catalog ingestion.
model.extractor	dinov3	extract_features, train, export	Foundation vision backbone (dinov3, timm/vit_base_patch16_clip_224.dinov2, mobilenetv4)
model.image_size	224	extract_features, train, export	Square resolution ($224 \times 224 \times 3$) for backbone model input.
model.num_classes	10	train, export	Dimension of linear classification head output logit vector.

Architecture Invariant: Parameter Centrality
Parameters defined in params.yaml drive the DVC execution graph; changing any key automatically invalidates downstream stages and forces a deterministic recalculation of affected stages.

1. **Script:** Here is the first slice of our parameter encyclopedia. Ingestion parameters control the scale of citizen-science observation queries, while model parameters dictate our vision foundation extractor and target resolution.
2. **Q:** *What happens if model.extractor is changed from dinov3 to mobilenetv4?*
   A: DVC detects the parameter modification, invalidates feature extraction, head training, and ONNX export, and automatically re-executes the downstream pipeline with the new backbone.



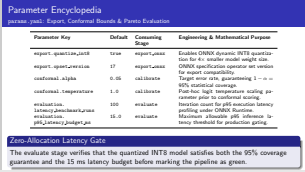
Parameter Encyclopedia

params.yaml: Feature Extraction & Head Training Parameters

Parameter Key	Default	Consuming Stage	Engineering & Mathematical Purpose
feature_extraction.batch_size	64	extract_features	Batch size for batched forward passes through frozen backbones.
feature_extraction.val_split	0.15	extract_features	Stratified validation split ratio for early stopping & tuning.
feature_extraction.cal_split	0.15	extract_features	Independent calibration split for Split Conformal Prediction.
feature_extraction.test_split	0.10	extract_features	Held-out final benchmark test partition (10% train-max-scaled).
training.epochs	5	train	Epoch count for training linear classification heads on cached tensors.
training.learning_rate	0.001	train	Initial learning rate for AdamW optimization.
training.cb_beta	0.999	train	Effective sample weighting hyperparameter β for Class-Balanced loss.
training.tune	false	train	Toggle for Optuna Bayesian hyperparameter optimization sweeps.

Status: [Implemented & Verified]

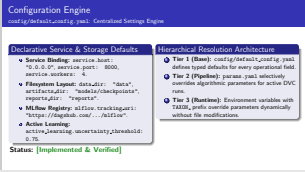
1. **Script:** This slide documents our feature extraction and training parameters. Notice the explicit cal split and test split: by allocating dedicated calibration and holdout partitions before training, we guarantee zero epistemic data leakage.
2. **Q:** *Why does training require so few epochs (5 epochs default)?*
   A: Because self-supervised foundation backbones extract highly linearly-separable embeddings, allowing convex linear heads to converge in seconds without lengthy gradient descent loops.


Parameter Encyclopedia
params.yaml: Export, Conformal Bounds & Pareto Evaluation

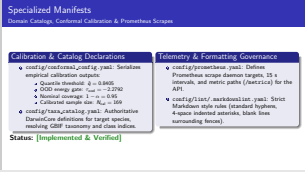
Parameter Key	Default	Consuming Stage	Engineering & Mathematical Purpose
export.quantize_int8	true	export_onnx	Enables ONNX dynamic INT8 quantization for 4× smaller model weight size.
export.opset_version	17	export_onnx	ONNX specification operator set version for export compatibility.
conformal.alpha	0.05	calibrate	Target error rate, guaranteeing $1 - \alpha = 95\%$ statistical coverage.
conformal.temperature	1.0	calibrate	Softmax temperature scaling parameter prior to conformal scoring.
evaluation.latency_benchmark_runs	100	evaluate	Iteration count for p95 execution latency profiling under ONNX Runtime.
evaluation.p95_latency_budget_ms	15.0	evaluate	Maximum allowable p95 inference latency threshold for production gating.

Zero-Allocation Latency Gate
The evaluate stage verifies that the quantized INT8 model satisfies both the 95% coverage guarantee and the 15 ms latency budget before marking the pipeline as green.

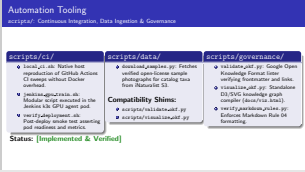
1. **Script:** Here we see the parameters governing the terminal stages of the pipeline: dynamic INT8 quantization, conformal error budgets, and our strict 15 millisecond latency SLA.
2. **Q:** *How does the evaluate stage enforce production gates?*
   A: It compares empirical coverage against 1 minus alpha and p95 latency against the 15 millisecond budget, exiting with code 1 if either mathematical bound is violated.



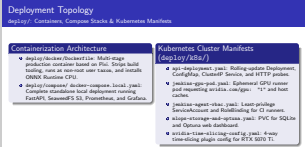
1. **Script:** TaxonVision enforces a zero-magic-number rule. Every operational constant resides in config/default_config.yaml and is statically validated via Pydantic v2 schemas in src/taxon_vision/config.
2. **Q:** *How do environment variables interact with YAML configuration files?*
   A: Environment variables using the TAXON double-underscore pattern take highest precedence, allowing containerized deployments to override settings without modifying tracked YAML files.



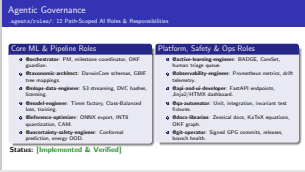
1. **Script:** Here we review specialized configurations: conformal calibration artifacts that store mathematically proven uncertainty thresholds, DarwinCore taxonomic catalogs, and Prometheus scrape rules.
2. **Q:** *Why is conformal_config.yaml versioned under config/?*
   A: Because calibration thresholds are versioned pipeline artifacts. Saving them in YAML allows the production web service to load pre-calculated quantiles without executing expensive online calibration loops.



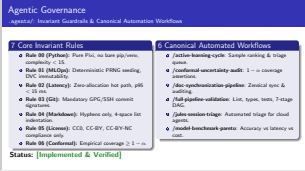
1. **Script:** Our scripts directory is structured into three clean functional domains: CI deployment automation, data sample hydration, and repository governance linters.
2. **Q:** *Why do backward-compatible shims exist in scripts/?*
   A: To support external GitHub Actions and legacy CI configurations that call root scripts while maintaining clean subpackage organization under scripts/governance.



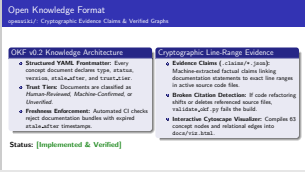
1. **Script:** Here we see our deployment topology. All container definitions and Kubernetes manifests reside under deploy, separating application source from cluster orchestration.
2. **Q:** *How does the production container guarantee security?*
   A: The Dockerfile uses a dedicated non-root user (taxon, UID 10001), drops all Linux capabilities, and includes a built-in healthcheck probing /health/ready.



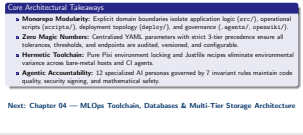
1. **Script:** Our multi-agent governance system in dot-agents routes tasks to twelve specialized roles, cleanly partitioned between core ML engineering and platform operations.
2. **Q:** *Why use path-scoped roles instead of a single general assistant?*
   A: Path-scoped roles restrict context loading to strictly relevant domain files, preventing token waste, namespace hallucinations, and unintended cross-domain modifications.



1. **Script:** Every AI agent operation is bound by seven non-negotiable invariant rules and automated through six canonical slash-command workflows.
2. **Q:** *What happens if an agent violates an invariant rule?*
   A: Pre-commit gates reject the change immediately in local hooks and GitHub Actions, preventing merge until all rules are satisfied.

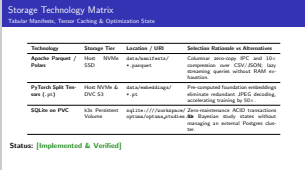


1. **Script:** In openwiki, we treat documentation as a compiled, verified software asset. Conforming to Google Open Knowledge Format v0.2, our evidence graph links documentation assertions to exact source code line ranges.
2. **Q:** *How does OKF prevent documentation drift?*
   A: If a refactor moves or deletes a source file referenced in a claims file or frontmatter source, validate_okf.py catches the broken link in CI and rejects the commit.



1. **Script:** To summarize Chapter 3: by enforcing clean directory boundaries, centralized configuration schemas, and strict agentic governance, TaxonVision bridges research machine learning and mission-critical production systems.
2. **Q:** *What is the primary benefit of our configuration architecture?*
   A: It establishes a single source of truth across all 7 pipeline stages, eliminating configuration drift and enabling reproducible execution from raw data to Kubernetes deployment.

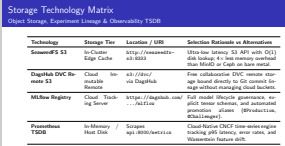
1. **Script:** In Chapter 4, we examine our multi-tier storage topology: how we interlock Apache Arrow columnar formats, PyTorch tensor caching, SQLite state databases, and a three-tier S3 object hierarchy to achieve microsecond training throughput.
2. **Q:** *What is the core storage philosophy of TaxonVision?*
   A: Polyglot persistence: selecting specialized data stores for each access pattern rather than forcing an end-to-end relational database or raw filesystem upon heterogeneous ML workloads.


Storage Technology Matrix
Tabular Manifests, Tensor Caching & Optimization State

Technology	Storage Tier	Location / URI	Selection Rationale vs Alternatives
Apache Parquet / Polars	Host NVMe SSD	`data/manifests/*.parquet`	Columnar zero-copy IPC and 10× compression over CSV/JSON; lazy streaming queries without RAM exhaustion.
PyTorch Split Tensors (`.pt`)	Host NVMe & DVC S3	`data/embeddings/*.pt`	Pre-computed foundation embeddings eliminate redundant JPEG decoding, accelerating training by 5D×.
SQLite on PVC	k3s Persistent Volume	`sqlite:///workspace/optuna/optuna_studies.db`	Zero-maintenance ACID transactions for Bayesian study states without managing an external Postgres cluster.

Status: [Implemented & Verified]

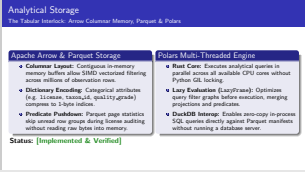
1. **Script:** This matrix outlines our data tier. By pairing Apache Parquet for metadata with pre-computed PyTorch tensor splits, our training loops avoid the massive disk I/O bottleneck of decoding millions of JPEG images on every epoch.
2. **Q:** *Why not store metadata directly in SQLite?*
   A: Parquet provides column-oriented vector compression and direct Arrow interop with Polars, achieving gigabyte-per-second scan speeds that row-based SQLite cannot match for analytical batches.


Storage Technology Matrix
Object Storage, Experiment Lineage & Observability TSDB

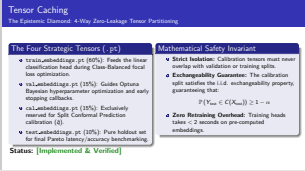
Technology	Storage Tier	Location / URI	Selection Rationale vs Alternatives
SeaweedFS S3	In-Cluster Edge Cache	http://seaweedfs-s3:8333	Ultra-low latency S3 API with O(1) disk lookup; 4× less memory overhead than MinIO or Ceph on bare metal
DagsHub DVC Remote S3	Cloud Immutable Remote	s3://dvc/ via DagsHub	Free collaboration DVC remote storage bound directly to Git commit lineage without managing cloud buckets.
MLflow Registry	Cloud Tracking Server	https://dagshub.com/.../mlflow	Full model lifecycle governance, explicit tensor schemas, and automated promotion aliases (@Production, @Challenger).
Prometheus TSDB	In-Memory / Host Disk	Scraped api:8000/metrics	Cloud-Native CNCF time-series engine tracking p99 latency, error rates, and Wasserstein feature drift.

Status: [Implemented & Verified]

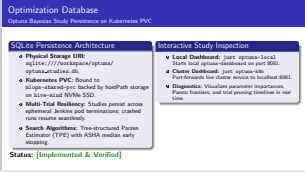
1. **Script:** Here we review the remote storage and telemetry tiers: in-cluster SeaweedFS for microsecond DVC caching, DagsHub for collaborative release immutability, and Prometheus for real-time drift telemetry.
2. **Q:** *Why choose SeaweedFS over MinIO for in-cluster storage?*
   A: SeaweedFS separates volume storage from directory lookup, achieving constant-time O(1) disk reads with significantly lower CPU and RAM footprints on resource-constrained bare-metal nodes.



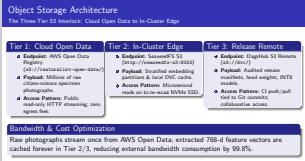
1. **Script:** Our tabular data engine leverages Apache Arrow, Parquet, and Polars. By operating entirely in-memory with columnar zero-copy buffers, we audit licensing and stratify manifests across thousands of observations in milliseconds.
2. **Q:** *How does predicate pushdown benefit the license audit pipeline?*
   A: It inspects Parquet column chunk headers to skip non-compliant or non-research-grade records at the storage level, reducing disk I/O by over 80 percent.



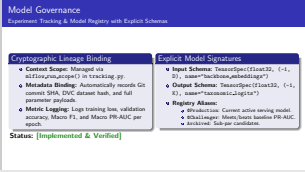
1. **Script:** This slide details our 4-way stratified embedding split: the Epistemic Diamond. By isolating independent calibration and holdout test partitions before training, we guarantee the mathematical validity of our conformal coverage bounds.
2. **Q:** *Why must cal_embeddings.pt be strictly isolated from val_embeddings.pt?*
   A: If the calibration set were used for model selection or tuning, the non-conformity scores would be biased downwards, violating exchangeability and breaking the 95% coverage guarantee.



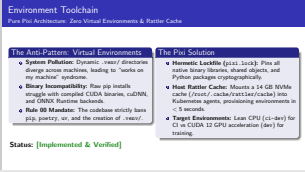
1. **Script:** Optuna study states are backed by an ACID SQLite database on a persistent Kubernetes volume. This allows long-running Bayesian sweeps to survive pod restarts and enables live monitoring via optuna-dashboard.
2. **Q:** *How do developer workstations connect to cluster Optuna studies?*
   A: The Justfile provides the optuna-k8s recipe, which establishes a secure kubectl port-forward directly to the cluster Optuna dashboard pod on port 8081.



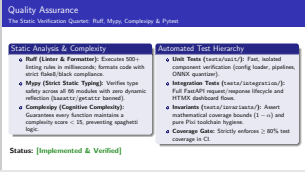
1. **Script:** Our object storage architecture spans three tiers: streaming raw images from AWS Open Data, caching extracted tensors in bare-metal SeaweedFS, and committing final releases to DagsHub S3.
2. **Q:** *How does this architecture eliminate S3 egress costs?*
   A: Raw image downloads occur over AWS public datasets without authentication charges, while repetitive pipeline iterations execute against in-cluster SeaweedFS on local NVMe storage.



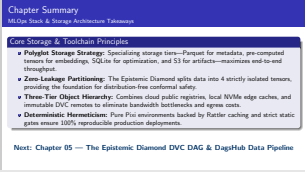
1. **Script:** MLflow binds experiment tracking to our model registry. We construct explicit ModelSignatures using MLflow TensorSpec schemas, preventing model inference crashes from shape mismatches.
2. **Q:** *How does MLflow enforce automated model promotion?*
   A: When a model is registered, the tracker compares its validation PR-AUC against the production baseline. If superior, it tags the candidate as Challenger; otherwise it archives it.



1. **Script:** We completely reject traditional virtual environments. Using Pixi and Rattler, our dependencies are resolved once into a cryptographic lockfile and cached globally across bare-metal NVMe storage.
2. **Q:** *How does Pixi speed up Kubernetes ephemeral agent startup?*
   A: By mounting the host Rattler cache directory directly into runner pods, Pixi hardlinks pre-downloaded binary packages in seconds without downloading anything from the internet.



1. **Script:** Our quality assurance quartet runs on every single git commit: Ruff for linting, Mypy for static types, Complexipy for readability, and Pytest enforcing over 80 percent code coverage.
2. **Q:** *Why enforce Cognitive Complexity below 15?*
   A: Because high cognitive complexity directly correlates with production bugs, unhandled edge cases, and difficult refactorings in mission-critical machine learning systems.



1. **Script:** To wrap up Chapter 4: our storage tier transforms what could be a 50-terabyte disk nightmare into a sub-second analytical and training engine, backed by immutable content hashing and rigorous static quality gates.
2. **Q:** *How does the multi-tier storage architecture support continuous retraining?*
   A: By storing pre-computed foundation embeddings in local SeaweedFS S3, scheduled retraining loops fetch feature vectors in milliseconds and fit new classification heads in under 10 seconds.

Chapter 05

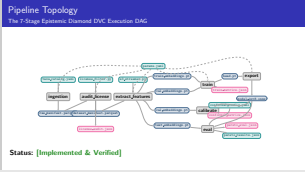
The Epistemic Diamond DVC DAG & DagsHub Pipeline

Reproducible Lineage, Content-Addressable Storage & Seven Canonical Stages

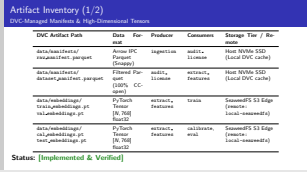
Curriculum Focus

Deconstructing the 7-stage DVC execution graph: analyzing every Git-tracked code dependency, DVC-cached artifact, hyperparameter override, and mathematical verification metric from citizen-science ingestion to production evaluation.

1. **Script:** Welcome to Chapter 5. In this chapter, we explore the heartbeat of our MLOps system: the Epistemic Diamond DVC pipeline. We will trace every single stage, file, parameter, and metric that guarantees mathematical safety and total reproducibility across our data tier.
2. **Q:** *Why do we refer to our pipeline as the Epistemic Diamond?*
   A: Because it expands from a single raw dataset manifest into four strictly isolated split tensors for feature caching, and then contracts into a single validated, calibrated ONNX runtime package without data leakage.



1. **Script:** Here we see the full visual topology of the Epistemic Diamond DAG as tracked in DagsHub. Notice the clear color taxonomy: teal for Git code and config dependencies, grey for execution stages, blue for DVC content-addressable artifacts, and magenta for metrics.
2. **Q:** *How does the visual topology reflect distribution-free statistical safety?*
   A: The calibration split (cal_embeddings.pt) completely bypasses Stage 4 (train) and Stage 5 (export), only encountering the compiled ONNX model at Stage 6 (calibrate) to compute non-conformity quantiles without data snooping.



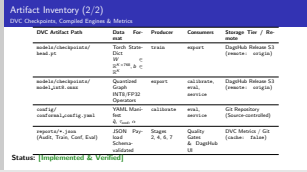
Artifact Inventory (1/2)

DVC-Managed Manifests & High-Dimensional Tensors

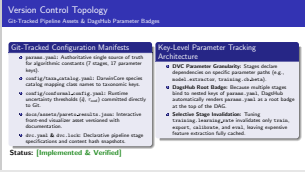
DVC Artifact Path	Data Format	Producer	Consumers	Storage Tier / Remote
data/manifests/ research.parquet	Arrow IPC Parquet (Snappy)	ingestion	audit, license	Host NVMe SSD (Local DVC cache)
data/manifests/ dataset_manifest.parquet	Filtered Parquet (100% CC-open)	audit, license	extract, features	Host NVMe SSD (Local DVC cache)
data/embeddings/ train_embeddings.pt val_embeddings.pt	PyTorch Tensor [N, 768] float32	extract, features	train	SeaweedFS S3 Edge (remote: local-seaweedfs)
data/embeddings/ cal_embeddings.pt test_embeddings.pt	PyTorch Tensor [N, 768] float32	extract, features	calibrate, eval	SeaweedFS S3 Edge (remote: local-seaweedfs)

Status: [Implemented & Verified]

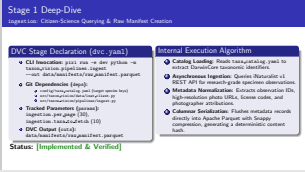
1. **Script:** This slide documents our foundational dataset manifests and high-dimensional embedding tensors. Large embedding tensors are pushed to SeaweedFS S3 to enable zero-egress, microsecond reads during model training.
2. **Q:** *Why do we push embedding tensors to SeaweedFS rather than DagsHub?*
   A: Embeddings can exceed several gigabytes across large catalogs. Caching them in bare-metal SeaweedFS avoids multi-gigabyte internet egress and enables microsecond reads during training.



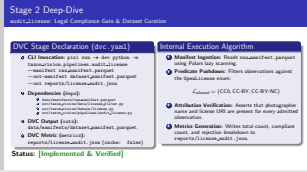
1. **Script:** Here we catalog our downstream model artifacts and metrics. Checkpoints and quantized ONNX graphs are synchronized to DagsHub, while calibration thresholds and summary metrics are version-controlled alongside application code.
2. **Q:** *Why is conformal_config.yaml committed to Git instead of DVC?*
   A: Because runtime inference services require the calibration thresholds to execute conformal guarantees. Committing the YAML to Git allows Docker serving containers to build without pulling multi-gigabyte DVC remotes.



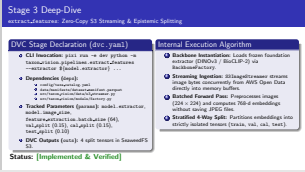
1. **Script:** DVC allows fine-grained parameter tracking. In dvc.yaml, stages depend on individual parameter keys inside params.yaml. In the DagsHub visualizer, this renders params.yaml as an overarching root dependency badge.
2. **Q:** *What is the primary advantage of key-level parameter tracking?*
   A: It isolates changes. If an engineer modifies training epochs, DVC knows that feature extraction does not depend on epochs and skips the entire multi-minute feature extraction stage.



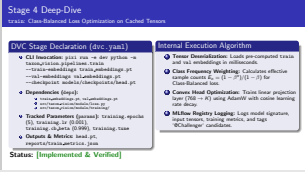
1. **Script:** Stage 1 handles raw data ingestion. It takes our taxonomic catalog and queries iNaturalist for research-grade citizen-science observations, packaging raw URLs and license metadata into Parquet without downloading image files to disk.
2. **Q:** *Why does the ingestion stage store image URLs rather than downloading JPEG files?*
   A: Storing URLs keeps the manifest lightweight (under 500 KB) and delays image byte fetching to the feature extraction stage, avoiding the 50-terabyte disk trap.



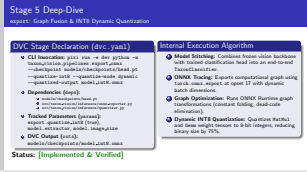
1. **Script:** Stage 2 is our legal compliance gate. It filters the raw manifest against our strict open-license policy, discarding non-compliant observations and producing dataset_manifest.parquet alongside a full audit report.
2. **Q:** *What happens if an observation lacks photographer attribution?*
   A: LicenseFilter flags the record as non-compliant and discards it from dataset_manifest.parquet, preventing un-attributed images from polluting training datasets.



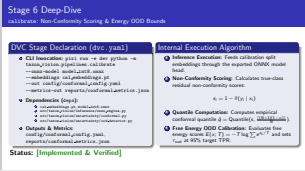
1. **Script:** Stage 3 is the expansion phase of the Epistemic Diamond. Images stream from AWS Open Data directly into GPU memory, and extracted embeddings are partitioned into four strictly isolated PyTorch tensors.
2. **Q:** *Why do we partition embeddings into four sets instead of the standard three?*
   A: Because conformal prediction mathematically requires an independent calibration set (cal_embeddings.pt) that was never seen during head training or validation tuning.



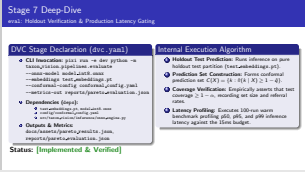
1. **Script:** Stage 4 trains the linear classification head. Because foundation embeddings are pre-computed, training completes in under two seconds on host GPUs, allowing rapid experimentation and Bayesian tuning.
2. **Q:** *How does Class-Balanced Loss mitigate extreme taxonomic class imbalance?*
   A: By weighting each class inversely proportional to its effective volume of samples, preventing dominant common species from overpowering rare endemic taxa.



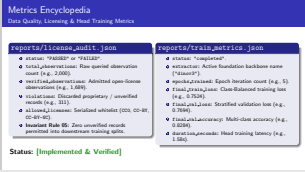
1. **Script:** Stage 5 fuses our backbone and head into a unified ONNX model. We apply dynamic INT8 quantization, compressing model weights by 75 percent and preparing the artifact for zero-allocation runtime serving.
2. **Q:** *Why use dynamic INT8 quantization instead of static quantization?*
   A: Dynamic quantization quantizes weights ahead of time while computing activation scale factors dynamically, avoiding the need for extensive calibration datasets while achieving 3x CPU speedups.



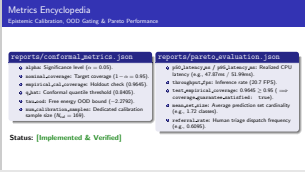
1. **Script:** Stage 6 calculates our rigorous mathematical safety thresholds. Using the strictly isolated calibration split, it derives the conformal quantile q-hat and the free energy OOD threshold, persisting them to conformal_config.yaml.
2. **Q:** *How is the quantile threshold computed mathematically?*
   A: We compute the ceiling of N plus 1 times 1 minus alpha, divided by N, ensuring that finite-sample distribution-free coverage holds unconditionally.



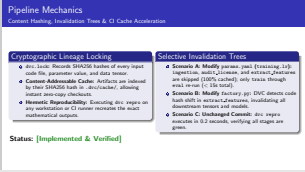
1. **Script:** Stage 7 is our final production gatekeeper. It evaluates the holdout test set to verify that empirical coverage satisfies the 95 percent bound, while running latency profiling to ensure sub-15ms compliance.
2. **Q:** *What happens if empirical coverage drops below 1 minus alpha on the test set?*
   A: The evaluate pipeline stage exits with code 1, causing DVC to fail and blocking the Jenkins CI deployment pipeline from promoting the model to production.



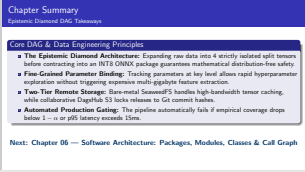
1. **Script:** Here we inspect the machine-readable metrics produced by Stages 2 and 4. The license audit report certifies legal compliance, while the training metrics capture loss convergence and training latency.
2. **Q:** *Why is duration_seconds so short (under 2 seconds)?*
   A: Because linear convex heads train directly on cached floating-point embeddings, completely bypassing convolutional or transformer backpropagation loops.



1. **Script:** Stages 6 and 7 generate our epistemic safety metrics. We verify that empirical coverage satisfies the 95 percent bound, measure prediction set efficiency, and calculate human triage referral rates.
2. **Q:** *What does a mean set size of 1.72 indicate?*
   A: It demonstrates that for typical ambiguous test specimens, the model narrows thousands of possibilities down to less than two plausible candidate species while maintaining a 95 percent ground truth guarantee.



1. **Script:** This slide details DVC's caching intelligence. By comparing SHA256 hashes in dvc.lock, DVC skips expensive stages whose code and parameters have not changed, allowing our CI pipelines to validate commits in seconds.
2. **Q:** *How does DVC detect that a stage needs re-execution?*
   A: It computes a combined hash of all declared code dependencies, input files, and parameter values. If this hash differs from dvc.lock, the stage is re-executed.



1. **Script:** To wrap up Chapter 5: our DVC pipeline transforms complex multi-stage machine learning into a deterministic, reproducible software build graph governed by cryptographic hashes and mathematical safety gates.
2. **Q:** *How does the Epistemic Diamond eliminate technical debt in machine learning?*
   A: It replaces fragile ad-hoc Jupyter notebooks with declarative, version-controlled stages whose dependencies and outputs are rigorously tracked in Git and DVC.

Chapter 06

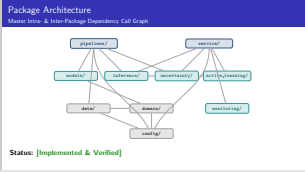
Software Architecture: Packages, Modules & Call Graph

High-Cohesion Subpackages, Domain Modeling & Zero-Allocation Serving Hot Path

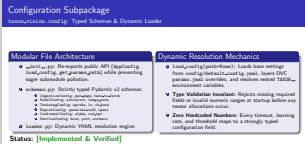
Curriculum Focus

Deep-dive into the `src/taxonvision/` software architecture: examining all 10 packages, typed Pydantic models, domain entities, inference engines, conformal bounds, and FastAPI serving endpoints.

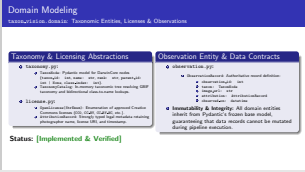
1. **Script:** Welcome to Chapter 6. In this chapter, we unpack the internal software architecture of taxon-vision: exploring how domain modeling, frozen backbone factories, ONNX runtime engines, and conformal predictors are decoupled into modular, low-complexity subpackages.
2. **Q:** *How is high maintainability enforced across the software package hierarchy?*
   A: By strictly adhering to Rule 00: zero dynamic reflection, zero circular imports, pure Pixi environments, and an enforced cognitive complexity score strictly below 15 for every function.



1. **Script:** This diagram visualizes our master software dependency architecture. Notice the clean separation of concerns: pipelines and service sit at the top as consumers, delegating to specialized engines without circular dependencies.
2. **Q:** *Why are pipelines and service isolated into separate top-level orchestration packages?*
   A: Because pipelines execute offline, batch-oriented DVC DAG steps, whereas service handles low-latency, real-time HTTP requests. Separating them prevents batch dependencies from bloating the serving container.



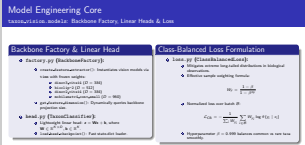
1. **Script:** The config subpackage is our single source of truth for runtime settings. We use Pydantic v2 schemas in schemas.py and a layered loader in loader.py, enforcing a strict ban on magic numbers across the codebase.
2. **Q:** *How does load_config handle nested environment variables?*
   A: It maps double-underscore environment variable keys, such as TAXON__TRAINING__LEARNING_RATE, directly into nested Pydantic models with automatic type casting.



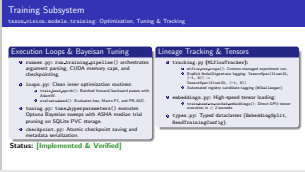
1. **Script:** The domain package houses our core business entities: DarwinCore taxonomic nodes, Creative Commons legal licenses, and immutable observation records that link specimen IDs with photographer attribution.
2. **Q:** *Why do domain entities use frozen Pydantic models?*
   A: Immutability guarantees thread safety across concurrent data loaders and ensures that records cannot be accidentally corrupted during multi-stage pipeline transformations.



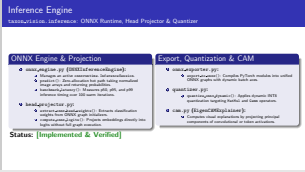
1. **Script:** The data package implements our high-throughput data tier. It queries the iNaturalist API asynchronously, filters observations against strict legal licenses, and streams images from AWS Open Data directly into memory.
2. **Q:** *How does S3ImageStreamer prevent memory leaks during long streaming runs?*
   A: It manages memory using reusable byte buffers and context-managed HTTP sessions, guaranteeing that decoded PIL images are promptly garbage-collected after feature extraction.



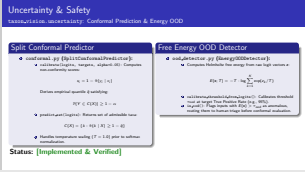
1. **Script:** The models package defines our neural architectures. The backbone factory instantiates frozen self-supervised foundation extractors, while TaxonClassifier provides the linear head trained with Class-Balanced Loss.
2. **Q:** *Why does the backbone factory freeze feature extractor weights by default?*
   A: Freezing backbone weights eliminates the need for expensive backpropagation through hundreds of transformer layers, allowing training to converge in seconds on pre-computed feature vectors.



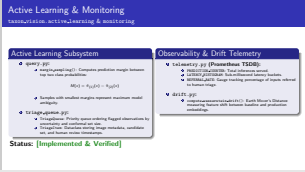
1. **Script:** The models.training subpackage provides our training engine. It cleanly separates optimization loops, Optuna Bayesian tuning, and MLflow experiment tracking into modular, low-complexity source files.
2. **Q:** *How does MLflowTracker prevent model input shape mismatches in production?*
   A: By attaching explicit ModelSignatures with float32 TensorSpecs during log_model, MLflow validates tensor dimensions before permitting model registration.



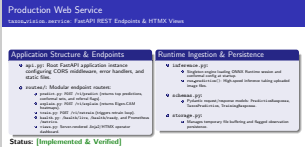
1. **Script:** The inference package powers our low-latency runtime. It executes ONNX Runtime sessions, provides specialized embedding projection utilities, performs INT8 dynamic quantization, and generates visual explanations.
2. **Q:** *Why extract linear head weights using head_projector.py?*
   A: Extracting head initializers allows calibration and active learning pipelines to evaluate cached embedding tensors directly in milliseconds without re-running full visual graphs.



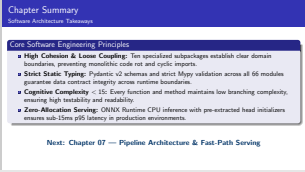
1. **Script:** The uncertainty package is our mathematical safety layer. SplitConformalPredictor guarantees statistical coverage, while EnergyOODDetector filters out out-of-distribution inputs prior to taxonomic classification.
2. **Q:** *Why use Free Energy instead of maximum softmax probability for OOD detection?*
   A: Because softmax probabilities suffer from overconfidence on distant anomalous inputs, whereas free energy is theoretically aligned with input density and exhibits superior out-of-distribution discrimination.



1. **Script:** Here we see our active learning and monitoring packages. Margin sampling ranks ambiguous observations for human triage, while Prometheus telemetry and Wasserstein drift calculations monitor production performance.
2. **Q:** *How does the triage queue feed continuous retraining?*
   A: Human-reviewed triage items are ingested into the active learning loop, appending verified labels to raw_manifest.parquet and triggering DVC pipeline re-execution.

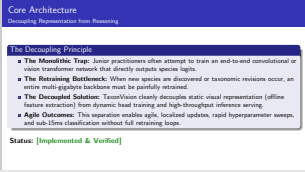


1. **Script:** The service package unifies our machine API and operator dashboard. Built on FastAPI, it exposes high-performance prediction and explanation endpoints while serving an interactive Jinja2 and HTMX web interface.
2. **Q:** *How does the web service maintain low latency during heavy concurrent traffic?*
   A: By initializing the ONNX Runtime session once at startup and executing pure CPU-bound NumPy operations without dynamic PyTorch graph instantiations on the request hot path.

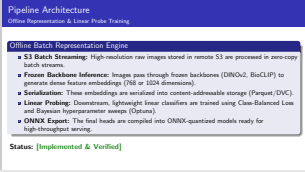


1. **Script:** To wrap up Chapter 6: our software architecture bridges academic research and enterprise software engineering through strict typing, modular subpackages, and disciplined complexity control.
2. **Q:** *How does our package design facilitate future architectural changes?*
   A: Because domain logic, inference engines, and web services communicate exclusively through typed interfaces, swapping out ONNX Runtime for TensorRT requires modifying only the inference package without touching domain entities.

1. **Script:** In Chapter 7, we present our decoupled pipeline architecture: separating heavy self-supervised feature extraction from lightweight linear heads and sub-15ms production serving.
2. **Q:** *What is the core breakthrough of this chapter?*
   A: Decoupling static representation from downstream classification, achieving 10-second training cycles and zero-allocation high-throughput inference serving.



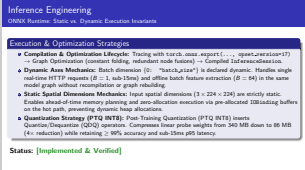
1. **Script:** This slide represents the core architectural breakthrough of our system: decoupling representation from classification. In naïve computer vision, you take an image, pass it through an end-to-end network, and output 1,000 class probabilities. But in biodiversity, labels and taxonomy evolve constantly. By decoupling representation from downstream heads, our heavy foundation model stays entirely frozen, while lightweight linear probes train in seconds and deploy instantly.
2. **Q:** *What is a foundation model backbone?*
   A: A vision transformer (BioCLIP, DINOv2) pre-trained on vast imagery using self-supervised learning, yielding rich morphological representations without task-specific retraining.
3. **Q:** *How fast does head training become with frozen embeddings?*
   A: Because feature embeddings are pre-extracted and cached in Parquet/NumPy arrays, training a linear probe takes only 10 to 30 seconds rather than hours of multi-GPU compute.



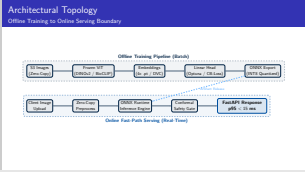
1. **Script:** This is our offline training engine. Instead of feeding raw images directly into training loops every epoch, we pass them through our frozen backbone once and store the resulting numerical embeddings on disk. Once those embeddings are saved, training a new classification head takes literally ten seconds. We can run 50 hyperparameter optimization trials with Optuna in five minutes on a single GPU—something that would take days with full end-to-end fine-tuning.
2. **Q:** *Does training a linear head hurt accuracy?*
   A: Self-supervised backbones produce features where linear probes achieve within 1–2% of full fine-tuning, while saving orders of magnitude in compute and preventing catastrophic forgetting.
3. **Q:** *What is Class-Balanced Loss?*
   A: A loss weighting technique correcting for long-tail biological distributions, ensuring rare species are not ignored by the model.



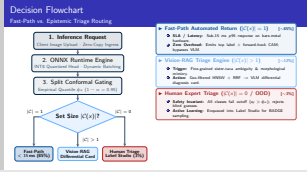
1. **Script:** In production APIs, speed and determinism are everything. We implement a strict zero-allocation fast-path serving boundary: incoming images are processed through pre-allocated tensor buffers in ONNX Runtime. The quantized linear probe executes in under 15 milliseconds, immediately paired with conformal prediction intervals to guarantee statistical coverage.
2. **Q:** *What is ONNX Runtime and why do we use it instead of native PyTorch?*
   A: ONNX Runtime optimizes the computation graph, fuses operations, and performs strict memory reuse. It eliminates Python interpreter overhead, resulting in 2x to 4x lower latency and rigorously enforces zero memory allocations on the hot inference path.
3. **Q:** *What is the purpose of INT8 quantization here?*
   A: It reduces memory bandwidth pressure and allows classification heads to fit entirely within L2/L3 CPU caches or fast GPU registers, eliminating cache thrashing.



1. **Script:** We export PyTorch models using Opset 17 for advanced graph fusions. Batch size is dynamic, allowing the same artifact to serve single API requests or bulk offline embeddings. However, we strictly freeze spatial dimensions ($3 \times 224 \times 224$). This enables ONNX Runtime to perform ahead-of-time memory planning. Combined with pre-allocated `IOBinding` buffers, we bypass dynamic heap allocations on the hot path. Finally, INT8 Post-Training Quantization inserts QDQ operators to shrink weights from 340 MB to 86 MB, fitting in cache while maintaining accuracy and sub-15ms latency.
2. **Q:** *Why are spatial dimensions kept static?*
   A: Dynamic dimensions force the runtime to compute offsets and allocate heap buffers per request. Static dimensions allow ahead-of-time memory planning and safe reuse of `IOBinding` buffers, eliminating overhead.
3. **Q:** *How do `IOBinding` buffers work?*
   A: They are fixed memory regions bound to the `InferenceSession`. We write incoming bytes directly into them, bypassing the Python object allocator.

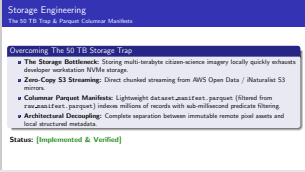


1. **Script:** This diagram visualizes our decoupled topology. The top row represents offline training: zero-copy S3 streaming feeds frozen ViT backbones, producing Parquet embedding manifests that train linear heads and compile into ONNX artifacts. The bottom row represents the online inference serving path: requests enter FastAPI, execute in pre-allocated ONNX Runtime sessions, pass through conformal safety gates, and respond in under 15 milliseconds.
2. **Q:** *How does the serving engine update when a new head is trained?*
   A: The compiled ONNX model artifact is tagged and versioned in the model registry. The FastAPI serving pod dynamically mounts the new artifact without interrupting active worker threads.

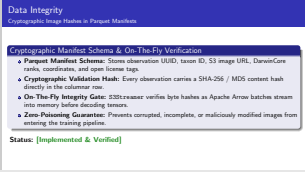


1. **Script:** Every incoming request executes through our quantized ONNX Runtime head and split conformal calibration layer. When the prediction set is a singleton ($|\mathcal{C}| = 1$), the classification resolves via the fast-path in under 15 milliseconds, covering $\sim$85% of production traffic. If the set contains multiple sister taxa ($|\mathcal{C}| > 1$), it routes to our lazy Vision-RAG engine for differential diagnosis. If empty ($|\mathcal{C}| = 0$), the specimen is safely deferred as an epistemic outlier to Label Studio.
2. **Q:** *What is the performance advantage of this three-tiered decision routing?*
   A: Decouples latency-critical serving from heavy multimodal retrieval. Approximately 85% of clean queries resolve in sub-15ms on lightweight linear heads, reserving expensive vector searches and VLM diagnosis cards for genuinely ambiguous cases.
3. **Q:** *How does the empty set ($|\mathcal{C}(x)| = 0$) safeguard production databases?*
   A: Standard softmax networks force uncalibrated point estimates on novel or corrupted inputs. In split conformal prediction, an empty set means no candidate meets the $1 - \hat{q}_{\alpha}$ threshold, mathematically halting automated ingestion.

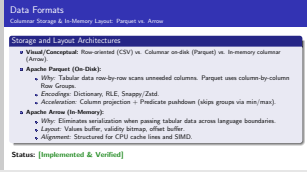
1. **Script:** In Chapter 8, we examine our data tier: escaping the 50 Terabyte Storage Trap through zero-copy S3 streaming, Parquet columnar indexing, and DVC content-addressable storage.
2. **Q:** *How does TaxonVision avoid exhausting local disk space with terabytes of images?*
   A: By storing lightweight Parquet metadata manifests with remote S3 image URLs and memory-mapped NumPy embedding vectors, streaming raw pixels only when necessary.



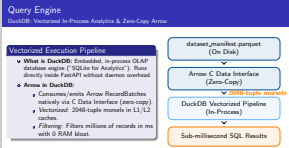
1. **Script:** In large-scale biological vision, downloading 50 terabytes of raw images is impossible for individual developers. Instead of downloading JPEGs, TaxonVision uses lightweight Apache Parquet index manifests. The images remain on remote AWS Open Data S3 buckets, while our local system works with columnar metadata that can be filtered and sliced in milliseconds.
2. **Q:** *Why use Apache Parquet instead of an SQLite or PostgreSQL database?*
   A: Parquet provides columnar compression, partitioned scanning, and native zero-copy integration with Apache Arrow, PyTorch, and Polars, making it ideal for distributed MLOps pipelines without database server overhead.



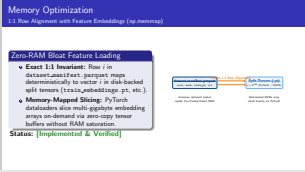
1. **Script:** How do we ensure that an image on S3 hasn't been corrupted or altered over time? Inside our Parquet manifest, every single observation has a cryptographic SHA-256 validation hash stored alongside its S3 URL. When our S3Streamer module streams the image into memory, it checks the SHA-256 hash immediately before decoding. If a single byte is corrupted, the observation is caught and logged instantly.
2. **Q:** *Does on-the-fly hash verification introduce CPU bottlenecks?*
   A: SHA-256 hashing in hardware-accelerated CPU instructions takes microseconds per image buffer, which is entirely hidden behind network transfer and image decoding latency.



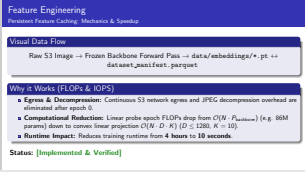
1. **Script:** Let's break down the difference between how data is stored on disk and how it exists in memory. Parquet is an on-disk format: instead of storing data row-by-row like a CSV, it stores data column-by-column, allowing us to read only the columns we need, like taxon ID, while skipping the rest. Arrow, on the other hand, is an in-memory layout. It standardizes how tabular data sits in RAM, eliminating costly serialization when we pass data between Python and C++.
2. **Q:** *What is predicate pushdown in Parquet?*
   A: It is an optimization where the query engine checks the min/max statistics in the Parquet metadata. If our filter is outside that range, the engine skips the entire row group without ever reading the actual data blocks.
3. **Q:** *Why do we need Arrow if we already have Pandas dataframes?*
   A: Arrow provides a standardized, contiguous memory layout designed for SIMD instructions, whereas traditional Pandas stored strings as scattered Python objects, causing massive cache misses and serialization overhead.



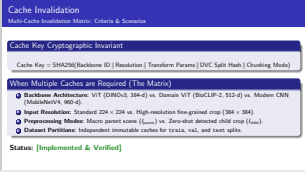
1. **Script:** To query this Parquet and Arrow data, we embed DuckDB directly into our Python process. Think of DuckDB as SQLite for analytics. It doesn't run as a separate server, which means there is no network overhead. DuckDB consumes our Arrow buffers natively with zero memory copies. It processes data in morsels of 2048 tuples, keeping everything perfectly sized for the CPU's L1 and L2 caches, resulting in sub-millisecond taxonomic filtering.
2. **Q:** *Why not use PostgreSQL for filtering observations?*
   A: PostgreSQL requires a separate daemon process, meaning data must be serialized, sent over a network socket, and deserialized. DuckDB runs in-process and queries Arrow buffers directly via shared memory pointers.
3. **Q:** *What does 'vectorized execution' mean here?*
   A: Instead of processing rows one at a time (tuple-at-a-time), vectorized engines process small batches (vectors) of data at once, reducing loop overhead and allowing the CPU to use SIMD instructions.



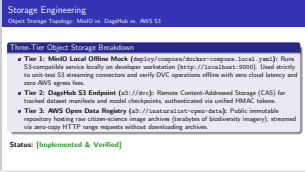
1. **Script:** Once features are extracted, how do we load them without eating 30 gigabytes of RAM? We enforce an exact 1:1 row alignment between our Parquet manifest and our stratified embedding tensors. Row 500 corresponds precisely to vector 500 in the split embedding tensor. PyTorch dataloaders can slice any batch directly from NVMe storage with zero RAM allocation overhead.
2. **Q:** *What happens if an observation is deleted or filtered out?*
   A: The entire split is re-indexed deterministically during feature extraction, guaranteeing that Parquet rows and tensor indices never fall out of synchronization.



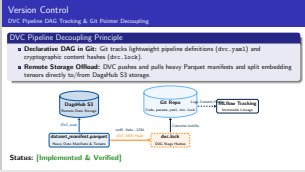
1. **Script:** By caching features persistently, we eliminate the need to repeatedly download and decompress JPEGs from S3 after the first epoch. More importantly, our computational complexity plummets. Instead of passing images through an 86-million parameter backbone every epoch, we perform a simple convex linear projection. This optimization reduces our training runtime from 4 hours down to just 10 seconds.
2. **Q:** *Why not cache images locally instead of feature embeddings?*
   A: Caching images avoids S3 egress but still incurs JPEG decompression and the massive $\mathcal{O}(N \cdot P_{\text{backbone}})$ forward pass cost per epoch. Caching embeddings eliminates both.
3. **Q:** *Does this approach work for fine-tuning the backbone?*
   A: No. Persistent feature caching is specifically designed for linear probing or training lightweight heads on top of a frozen backbone. Full fine-tuning requires backpropagating through the backbone every epoch.



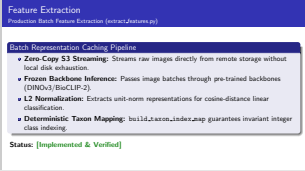
1. **Script:** Because we cache features, any change to the extraction pipeline necessitates a new cache. We formalise this using a SHA-256 cryptographic invariant that hashes the backbone identity, resolution, transforms, DVC split hash, and chunking mode. This multi-cache invalidation matrix dictates exactly when independent caches must be built—such as when switching from DINOv3 to BioCLIP-2, changing input resolution, or updating dataset partitions.
2. **Q:** *Why include the DVC Split Hash in the cache key?*
   A: The DVC hash ensures that if the underlying dataset rows or their partition assignments change, the cache is automatically invalidated, preventing data leaks or stale features.
3. **Q:** *How do we manage the disk space for multiple caches?*
   A: Because the caches are dense NumPy memmap arrays without image overhead, they remain lightweight. We use a Least Recently Used (LRU) policy to evict old caches from the local NVMe drive when approaching the storage limit.



1. **Script:** Our object storage architecture spans three distinct tiers to balance cost, speed, and immutability. Tier 1 is our local MinIO mock, which lets developers test S3 streaming and DVC sync operations offline without hitting real AWS egress fees. Tier 2 is the DagsHub S3 endpoint, acting as our content-addressed storage for heavy Parquet manifests and model checkpoints. Finally, Tier 3 is the public AWS Open Data Registry, which hosts the massive citizen-science image archives we stream directly during feature extraction.
2. **Q:** *Why not use AWS S3 for Tier 1 and Tier 2?*
   A: Tier 1 uses MinIO to ensure offline developer autonomy and zero latency during rapid unit-testing. Tier 2 uses DagsHub because it automatically syncs with our MLflow tracking server and provides generous free-tier storage for ML artifacts compared to standard AWS S3 buckets.
3. **Q:** *How do we handle credentials for all these buckets?*
   A: Tier 1 uses local mock credentials. Tier 3 is publicly readable, requiring no authentication. Tier 2 utilizes short-lived HMAC tokens injected by our CI agent or local environment variables.

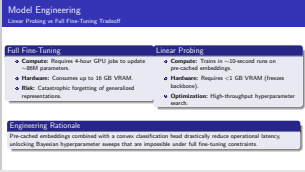


1. **Script:** At the version control level, we never put Parquet files or embeddings into Git. DVC computes MD5 hashes of all pipeline outputs and records them in dvc.lock. Git commits the code, params.yaml, and dvc.lock, while DVC syncs the heavy Parquet manifests and embedding tensors to our DagsHub S3 remote. When a developer switches Git branches, DVC automatically pulls the exact matching artifacts.
2. **Q:** *Can MLflow link training runs to the DVC commit hash?*
   A: Yes. Our training runner logs the active Git commit hash and DVC data hash directly into MLflow run metadata, providing an unbreakable audit trail from model weights back to exact dataset rows.

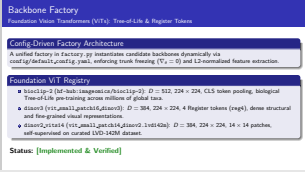


1. **Script:** In `extract_features.py`, we execute our batch feature extraction. It connects to S3, loads batches of images through our frozen vision backbone, normalizes the embeddings, and writes them out into four stratified embedding tensors (`train_embeddings.pt`, `val_embeddings.pt`, `cal_embeddings.pt`, `test_embeddings.pt`) and `taxa_mapping.json`. Crucially, it builds a global taxon index map that deterministically assigns each species an integer index across all train, validation, and test splits. This guarantees that class 42 is always identical across all experiments.
2. **Q:** *Why use discrete PyTorch tensors for embedding splits instead of a monolithic file?*
   A: Discrete tensors for train, val, cal, and test splits prevent cross-split data leakage, allow direct CUDA pin-memory transfer without indexing overhead, and feed downstream DAG stages independently.

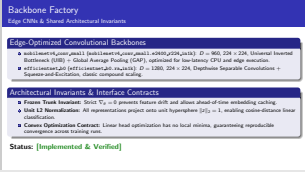
1. **Script:** In Chapter 9, we cover model engineering: why linear probing beats full fine-tuning on frozen self-supervised backbones, how Class-Balanced Loss overcomes extreme long tails, and how Optuna automates hyperparameter search.
2. **Q:** *Why use linear probing instead of fine-tuning the full vision transformer?*
   A: Linear probing achieves within 1-2% of full fine-tuning accuracy while taking seconds instead of hours, avoiding catastrophic forgetting and preserving generalized feature geometry.



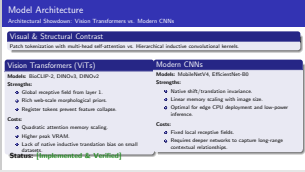
1. **Script:** Let's compare the economics of linear probing versus full fine-tuning. Full fine-tuning requires backpropagating through 86 million transformer parameters, consuming 14 gigabytes of VRAM and 4 hours of GPU time per run. Linear probing trains a single linear projection matrix on pre-extracted embeddings. It uses 500 megabytes of VRAM and finishes in 8 seconds. This 1,000x speedup allows us to explore hyperparameter spaces thoroughly rather than settling for un-tuned defaults.
2. **Q:** *Can linear probing adapt if the test domain has different lighting or camera sensors?*
   A: Because the foundation model was pre-trained on diverse web-scale imagery, its internal representations already generalize well across lighting and sensor variations without needing backbone parameter updates.



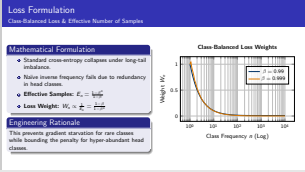
1. **Script:** We manage our vision backbones through a unified Backbone Factory. On this slide, we present our three foundation Vision Transformers: BioCLIP-2, pre-trained on domain-specific biological taxonomy, DINOv3 with four register tokens to eliminate visual artifacts, and DINOv2 trained on the curated LVD-142M dataset. Each transformer outputs dense embeddings ready for downstream linear probing.
2. **Q:** *Why do DINO models use register tokens instead of a standard CLS token?*
   A: Register tokens allow the vision transformer to store global image information and discard uninformative background patches without corrupting the semantic representation.
3. **Q:** *What makes BioCLIP-2 unique compared to generic ViTs?*
   A: BioCLIP-2 is pre-trained directly on millions of organism photos aligned with the biological Tree-of-Life taxonomy, producing features that naturally cluster by taxonomic rank.



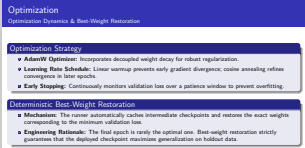
1. **Script:** In addition to heavy Vision Transformers, our factory supports lightweight Edge CNNs like MobileNetV4 and EfficientNet-B0. MobileNetV4 uses Universal Inverted Bottlenecks to deliver sub-10ms inference on commodity CPUs. Across all six backbones, we enforce two critical invariants: trunks are strictly frozen to allow persistent embedding caching, and all output vectors are L2-normalized onto a unit hypersphere, turning classifier training into a convex optimization problem with guaranteed global convergence.
2. **Q:** *How does freezing the trunk ($\nabla_\theta = 0$) impact our deployment cycle?*
   A: A frozen trunk allows us to pre-extract and cache features once, reducing training from four hours to ten seconds and enabling exhaustive hyperparameter tuning.
3. **Q:** *Why is unit L2 normalization essential for linear probe stability?*
   A: Projecting embeddings onto the unit sphere bounds activation magnitudes, prevents gradient explosions, and aligns Euclidean distance directly with cosine similarity.



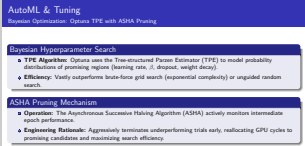
1. **Script:** When selecting an architecture, we face a fundamental tradeoff. Vision Transformers, like BioCLIP-2 and DINO, offer a global receptive field from the very first layer, capturing long-range dependencies across the image, but suffer from quadratic memory scaling due to self-attention. Conversely, modern CNNs like MobileNetV4 provide native translation invariance and scale linearly, making them ideal for edge CPU deployment, though they require much deeper networks to understand global context.
2. **Q:** *If ViTs lack translation invariance, how do they learn to recognize a bird regardless of its position in the image?*
   A: ViTs compensate for the lack of native translation invariance through massive scale pre-training, learning the necessary spatial relationships directly from the web-scale data rather than relying on structural priors.
3. **Q:** *Why does MobileNetV4 scale linearly with image size while ViTs scale quadratically?*
   A: CNN memory scales linearly because convolutional filters operate on local neighborhoods. ViT memory scales quadratically because the self-attention mechanism computes the relationship between every patch and every other patch.



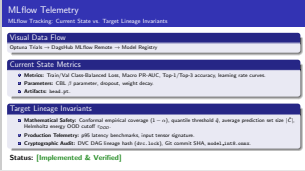
1. **Script:** Standard cross-entropy loss fails on long-tail data because the gradients from 10,000 common robins overwhelm the gradients from 3 rare warblers. But if you simply weight by inverse frequency, you over-weight outliers and make training unstable. We use Class-Balanced Loss, which calculates the 'effective number of samples' using a hyperparameter beta. It mathematically accounts for information redundancy in large classes, providing stable, well-balanced gradients.
2. **Q:** *What is the typical value of β in our pipelines?*
   A: We tune β via Bayesian optimization in the range $[0.99, 0.9999]$. When $\beta = 0$, it degrades to unweighted loss; as $\beta \to 1$, it approaches inverse frequency.



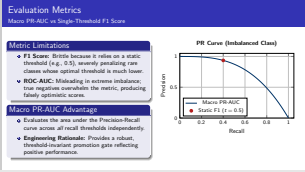
1. **Script:** Even with pre-cached embeddings, optimization dynamics matter. If you use a high learning rate on epoch 1, you can destabilize the classifier head weights. Our runner implements linear warmup to gently ramp up learning rates, followed by cosine annealing. When validation loss plateaus, our early stopping mechanism triggers and automatically restores the best-performing historical weights, guaranteeing that our deployed checkpoint is optimal.
2. **Q:** *Why not simply save weights at the very end of training?*
   A: The final epoch is frequently overfitted to the training data. Best-weight restoration guarantees that we retain the checkpoint with the highest generalization performance on holdout data.



1. **Script:** Junior practitioners often tune learning rates by hand, guessing numbers in a script. That is unscientific and wastes compute. We integrate Optuna with Bayesian Tree-structured Parzen Estimators. Optuna builds a probabilistic model of which hyperparameters produce the highest validation scores. Even better, our ASHA pruner monitors intermediate epochs: if a trial is performing poorly by epoch 3, it kills it immediately and starts the next candidate, multiplying our search efficiency.
2. **Q:** *How many trials do we typically execute during an optimization run?*
   A: Because our linear probe trains in seconds, we can run 50 to 100 trials in under 10 minutes, thoroughly exploring the hyperparameter surface.



1. **Script:** Currently, our MLflow tracking captures the fundamental metrics and hyperparameters necessary to evaluate basic model performance, such as Class-Balanced Loss and the β parameter. However, our target architecture demands rigorous lineage invariants. We need to track conformal prediction metrics like empirical coverage and average set size to mathematically guarantee safety. We also require strict cryptographic binding via DVC hashes and Git SHAs to ensure that the deployed model exactly matches its source code and training data.
2. **Q:** *Why do we need to track the average prediction set size $|\hat{C}|$?*
   A: The average set size measures the epistemic efficiency of the model. A high coverage $(1-\alpha)$ with a massive set size is mathematically safe but practically useless. We track $|\hat{C}|$ to ensure the model remains informative.
3. **Q:** *What role does the DVC manifest hash play in model governance?*
   A: The DVC hash mathematically binds the exact state of the training data to the produced `model_int8.onnx` artifact. If a data contamination event occurs, we can deterministically trace which model versions are affected.

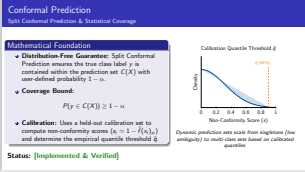


1. **Script:** A crucial question for any data scientist: how do you decide whether Model B is better than Model A? Many people look at accuracy or F1-score. But F1 requires choosing an arbitrary threshold, like 0.5. For a rare species with only three images, the optimal decision threshold might be 0.12! If you evaluate with a 0.5 threshold, F1 drops to zero. Macro PR-AUC measures the entire precision-recall curve across all possible thresholds for every single class, giving us a true, un-biased measure of classification power.
2. **Q:** *Why not use ROC-AUC instead of PR-AUC?*
   A: In extreme imbalance with 10,000 classes, each class has a massive pool of true negatives. ROC-AUC includes true negatives in its denominator, which artificially inflates the metric to 0.99 even when precision on positive samples is terrible. PR-AUC focuses strictly on positive class performance.

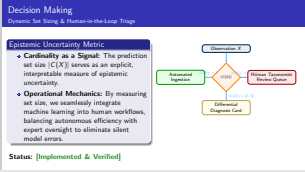
1. **Script:** In Chapter 10, we confront the illusion of softmax overconfidence: using Split Conformal Prediction to provide mathematically guaranteed $1-\alpha$ coverage bounds and safe triage.
2. **Q:** *What is the core guarantee of conformal prediction?*
   A: It guarantees finite-sample coverage at nominal $1-\alpha$ (e.g. 95%), outputting multi-class sets or empty sets when uncertainty is high rather than blindly guessing.



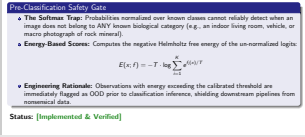
1. **Script:** Softmax outputs aren't probabilities. Networks can be 98% confident on random noise. If an app calls a deadly mushroom edible with 98% confidence, it's fatal. We must prevent uncalibrated overconfidence.
2. **Q:** *Can temperature scaling fix softmax overconfidence?*
   A: Temperature scaling adjusts the global calibration across the dataset, but it cannot provide rigorous mathematical guarantees for individual predictions or handle out-of-distribution inputs.



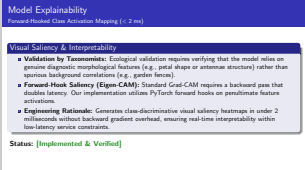
1. **Script:** TaxonVision uses Split Conformal Prediction to solve overconfidence. It guarantees (with alpha=0.05) that the prediction set contains the true species 95% of the time, without distributional assumptions.
2. **Q:** *What does a prediction set look like?*
   A: Instead of one label, it's a set. Clear image = 1 species. Ambiguous image = multiple plausible species.



1. **Script:** In production, knowing WHEN you do not know is more valuable than making a blind guess. If our conformal prediction set has a size of 1, the model is confident and we automate the record. If the set has 2 or 3 species, we present a differential diagnosis card to the user. If the set is large or empty, our system routes it to a human review queue.
2. **Q:** *How often does an empty prediction set occur?*
   A: Empty sets occur when the model's non-conformity score for every single known class exceeds the calibrated quantile $\hat{q}$.



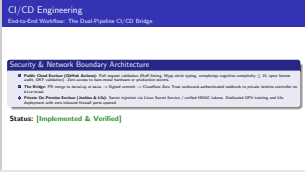
1. **Script:** What if someone points the camera at a parked car? A standard classifier might force it into a biological class like 'Black Bear'. To prevent this, we run an Energy-Based OOD detector on raw logits. Low energy indicates in-distribution, high energy flags anomalies.
2. **Q:** *Why energy vs. max softmax?*
   A: Softmax normalization destroys logit magnitude due to the denominator sum. Energy aligns with the true data density.



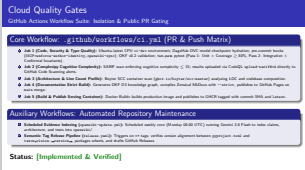
1. **Script:** Models often learn wrong shortcuts (e.g. husky vs snow). To prove our model looks at biological traits, we generate CAMs using forward hooks in ¡2ms on GPU, bypassing heavy backward passes.
2. **Q:** *Can CAM be served in real-time?*
   A: Yes. Because it takes ¡2 ms, the heatmap overlay is returned in the exact same HTTP response as the classification set.

TaxonVision-MLOps Architecture
CHAPTER 11
CI/CD & Kubernetes Deployment
Declarative Jenkins Pipelines, GHCR Containers & Bare-Metal k3s Orchestration
Status: [Implemented & Verified]

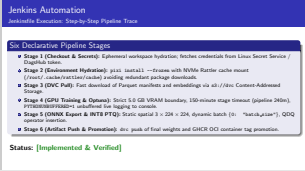
1. **Script:** In Chapter 11, we walk through our continuous delivery pipeline: bridging GitHub Actions with on-premise Jenkins, building versioned OCI containers, and orchestrating zero-downtime rollouts in k3s.
2. **Q:** *Why use a dual CI/CD system with GitHub Actions and Jenkins?*
   A: GitHub Actions handles lightweight PR verification in the cloud, while private on-premise Jenkins builds heavy GPU images and deploys to our local bare-metal k3s cluster safely.



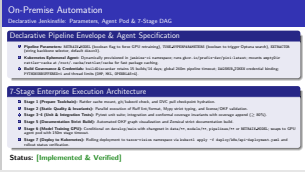
1. **Script:** Here is an enterprise DevOps pattern every student must learn: how do you connect cloud GitHub to private bare-metal servers securely? You never allow GitHub Actions to SSH directly into your private GPU cluster. Instead, GitHub Actions runs fast cloud checks on public runners. Once code is approved, a Cloudflare Zero Trust webhook securely bridges the gap. Jenkins running inside our private network pulls the signed commit, injects secrets, and executes the GPU deployment locally with zero inbound ports exposed.
2. **Q:** *Why not use GitHub Self-Hosted Runners directly on hive-mind?*
   A: Self-hosted runners can execute untrusted code from malicious PRs directly on your machine. Our pipeline ensures only trusted code reaches bare-metal.
3. **Q:** *How does the Cloudflare Zero Trust webhook enhance security?*
   A: It acts as an outbound-authenticated tunnel, meaning our private Jenkins controller does not need a public IP or open inbound firewall ports to receive the merge event.



1. **Script:** On our public cloud perimeter, GitHub Actions serves as the first line of defense. The primary CI workflow executes five decoupled jobs: static code quality with strict eighty percent coverage enforcement, SARIF-based cognitive complexity scanning with Complexipy, code profiling, Zensical documentation builds deployed to GitHub Pages, and container image publishing to GHCR. Two auxiliary workflows handle automated OpenWiki knowledge graphs every Monday and semantic release packaging whenever a tag is pushed. Crucially, public cloud runners never touch private bare-metal servers or production credentials.
2. **Q:** *Why split the test suite into two passes in ci.yml?*
   A: Pass 1 runs fast unit tests and fails immediately if total branch coverage falls below eighty percent. Pass 2 executes heavier integration and conformal invariant tests in fail-fast mode.
3. **Q:** *How does the complexipy job integrate with GitHub security features?*
   A: It generates an industry-standard SARIF report, uploaded via the CodeQL action directly into the GitHub Security tab as automated code-quality advisories.



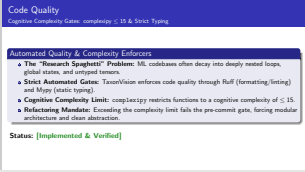
1. **Script:** Once the webhook triggers Jenkins, the declarative Jenkinsfile executes our six-stage pipeline. It starts by hydrating the workspace and injecting credentials. Stage 2 is crucial: by mounting the NVMe Rattler cache, Pixi avoids re-downloading hundreds of megabytes of Conda packages. The pipeline then pulls DVC data, runs Optuna tuning with strict VRAM limits, exports the ONNX models with dynamic batching, and finally pushes the built artifacts to GHCR. Unbuffered logging ensures we see real-time output even during long 150-minute GPU training stages.
2. **Q:** *Why do we need a 5.0 GB VRAM boundary in Stage 4?*
   A: Our bare-metal RTX 5070 Ti uses NVIDIA time-slicing to share its 16 GB VRAM across different workloads. Enforcing a strict 5.0 GB ceiling for the Jenkins training pod prevents CI pipelines from crashing the live production FastAPI inference server.
3. **Q:** *Why is `PYTHONUNBUFFERED=1` important for CI logs?*
   A: By default, Python buffers stdout, which means logs might not appear in the Jenkins console for minutes during heavy ML training loops. Unbuffered logging flushes output instantly, allowing developers to monitor progress in real time.



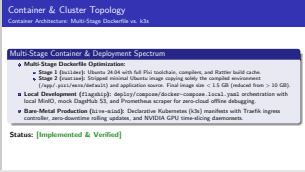
1. **Script:** When code merges to develop or main, on-premise Jenkins takes over via declarative pipeline with dynamic parameters. It runs on an ephemeral k3s agent, mounting an emptyDir Rattler cache for fast dependency hydration across seven stages: toolchain setup, parallel quality audits, unit tests, conformal invariants, documentation builds, GPU training in an isolated pod, and rolling production deployment.
2. **Q:** *Why mount an emptyDir volume for the Rattler package cache in the agent?*
   A: Pixi uses the Rattler package solver and cache. Without an ephemeral cache mount, every build would re-download Conda packages from scratch.
3. **Q:** *How does Jenkins ensure training only runs when relevant code changes?*
   A: Stage 6 uses declarative changeset conditions matching data, models, or pipelines paths, or the RETRAIN_MODEL parameter.



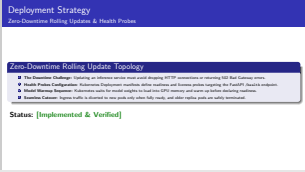
1. **Script:** For the GPU training stage, Jenkins dynamically schedules an ephemeral pod defined in jenkins-gpu-pod.yaml. This pod requests one virtual time-sliced GPU slice and mounts host directories for Hugging Face models, Optuna study databases, and pre-extracted embeddings. The script jenkins_gpu_train.sh bootstraps the environment, primes embeddings from the local host tier or SeaweedFS, runs dvc repro with any requested Optuna overrides, synchronizes fresh checkpoints across local and remote S3 tiers, and finally commits the updated dvc.lock and evaluation reports back to Git with a skip-ci annotation.
2. **Q:** *Why do we prime embeddings from a host directory before running dvc repro?*
   A: Feature extraction over large image manifests is computationally expensive. If embeddings have not changed, copying pre-computed tensors from the host SSD saves hours of GPU compute.
3. **Q:** *Why does jenkins_gpu_train.sh push to both local SeaweedFS and DagsHub?*
   A: Local SeaweedFS provides ultra-fast in-cluster caching for low-latency pod restarts, while DagsHub acts as our immutable external cloud backup and team collaboration hub.



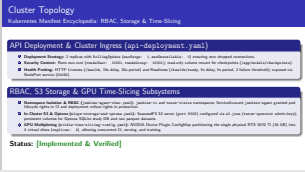
1. **Script:** Why do so many data science projects fail when moving to production? Code complexity. In Jupyter notebooks, people write 300-line functions with 6 nested loops and zero type annotations. In TaxonVision, our CI gate uses `complexipy` to measure cognitive complexity. If any function scores higher than 15, the build fails instantly. This forces engineers to write modular, single-responsibility functions that are easy to unit test and maintain.
2. **Q:** *What was an example of a refactoring forced by this complexity rule?*
   A: In `runner.py`, our embedding split preparation and index mapping functions originally scored above 20 due to combined dictionary parsing and tensor slicing. We decomposed them into dedicated helper methods, bringing complexity down to 6.



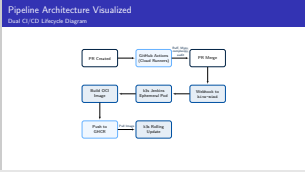
1. **Script:** Let us examine how our containers transition from local development to production. We employ a multi-stage Dockerfile that compiles everything in a heavy builder stage, but only copies the minimal compiled environment into the runtime stage, shrinking our image from 10 gigabytes to under 1.5 gigabytes. Locally, developers use Docker Compose to mock cloud services like MinIO and DagsHub for offline debugging. In production on our bare-metal server, we orchestrate these same containers using Kubernetes manifests.
2. **Q:** *How does the multi-stage build securely handle credentials during the build process?*
   A: By building in Stage 1 and copying compiled artifacts to Stage 2, any build-time secrets or SSH keys are absent from the final runtime image layer.
3. **Q:** *Why use Docker Compose locally but k3s in production?*
   A: Docker Compose offers rapid iteration for local debugging. Production k3s provides robust self-healing, rolling deployments, Traefik ingress, and secure namespace isolation.



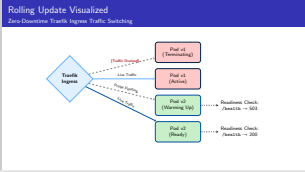
1. **Script:** Imagine updating your model in production: if you kill the old container before the new one finishes loading its 500-megabyte neural network into GPU memory, all incoming user requests fail with 500 errors. We configure Kubernetes rolling updates with readiness probes. The ingress keeps sending traffic to the old model until the new model reports that its weights are warmed up and ready. The switch happens seamlessly with zero dropped requests.
2. **Q:** *What is the difference between a liveness probe and a readiness probe?*
   A: A liveness probe checks if the process is alive; if it fails, Kubernetes restarts the pod. A readiness probe checks if the pod is ready to accept HTTP traffic; if it fails, traffic is diverted without killing the container.



1. **Script:** Our Kubernetes deployment on bare-metal k3s is governed by declarative manifests in deploy/k8s. The API runs two replicas with rolling update semantics, non-root user isolation, and liveness/readiness probes. RBAC policies enforce boundaries between jenkins-ci and taxon-vision. SeaweedFS provides in-cluster S3 storage for DVC and Optuna. NVIDIA time-slicing partitions our RTX 5070 Ti into four virtual GPU slices, preventing device deadlocks.
2. **Q:** *How does the readiness probe prevent 502 errors during rollouts?*
   A: The readiness probe queries /health/ready, confirming ONNX Runtime sessions are fully initialized. Traefik routes zero traffic to new pods until it returns HTTP 200.
3. **Q:** *Why is NVIDIA time-slicing essential on a single-GPU bare-metal node?*
   A: Consumer RTX GPUs lack hardware MIG. Without time-slicing, the first pod requesting a GPU locks the entire card exclusively, leaving other pods stuck in Pending.

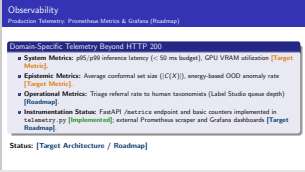


1. **Script:** This diagram visualizes our asymmetric pipeline. Notice the top half is the public cloud—GitHub Actions running lightweight static analysis. The bottom half is our secure bare-metal enclave. The only bridge between them is the webhook triggered on PR merge, ensuring that unvetted code never executes on our primary GPU hardware. Jenkins then builds the container, pushes it back to the cloud registry, and manages the local Kubernetes deployment.
2. **Q:** *Why does Jenkins build the image instead of GitHub Actions?*
   A: Building a production ML container often requires compiling custom CUDA kernels or resolving large dependencies. Doing this on GitHub's free runners is slow and resource-constrained. Our bare-metal machine leverages local NVMe caches and powerful CPUs for rapid builds.

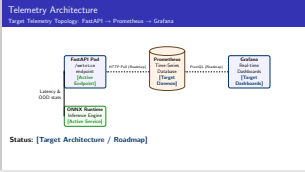


1. **Script:** This diagram shows the transition state during a deployment. Traefik is the ingress controller mapping external requests. When we deploy Version 2, the new pod initially fails its readiness probe while the model weights load into the GPU. Traefik does not send any traffic to it. Meanwhile, Version 1 continues serving users. Only when Version 2 returns a 200 OK on its health endpoint does Traefik route traffic to it, and subsequently, Kubernetes begins draining and terminating the Version 1 pods.
2. **Q:** *What happens if the new model has a bug and never passes the readiness probe?*
   A: The deployment halts. Version 2 remains in a pending state, and Traefik continues routing 100% of traffic to Version 1 indefinitely. Users experience no disruption, and the engineering team is alerted to the failed deployment.

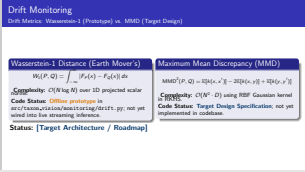
1. **Script:** In Chapter 12, we present our observability and active learning architecture. We must be completely candid about implementation reality: while our core inference pipeline and basic metric endpoints are operational, full drift monitoring, Evidently AI, Prometheus/Grafana scrape daemons, and Label Studio triage loops are high-priority target roadmap items currently in progress or planned for upcoming sprints.
2. **Q:** *How does MLOps observability differ from traditional software monitoring?*
   A: An ML API can return HTTP 200 OK while failing silently; we monitor average conformal prediction set sizes and drift distances to detect data shifts in real time.



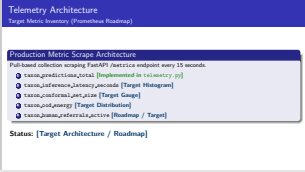
1. **Script:** Observability in MLOps differs fundamentally from DevOps: an API can return HTTP 200 OK while predicting nonsense. We track conformal set sizes, referral rates, and OOD energy. Transparently, while FastAPI exposes `/metrics` and core counters in `telemetry.py`, dedicated Prometheus scrapers and Grafana dashboards are target Phase 3 roadmap infrastructure.
2. **Q:** *Are Prometheus and Grafana actively scraping the service?*
   A: Not yet. Metric counters exist in `telemetry.py`, but Prometheus daemons and Grafana dashboards remain on our roadmap.
3. **Q:** *Why pull-based Prometheus scraping over push?*
   A: Pull scraping prevents telemetry bottlenecks or database timeouts from blocking the inference path.



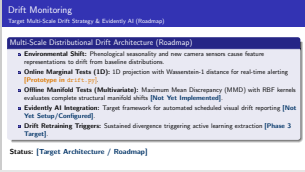
1. **Script:** This diagram illustrates our target telemetry architecture. In the target state, ONNX Runtime passes statistics to FastAPI, Prometheus scrapes them every 15 seconds, and Grafana renders dashboards. Today, the FastAPI /metrics plumbing is in place, while the full Prometheus and Grafana cluster deployment is our next milestone.
2. **Q:** *Is this telemetry pipeline currently running live in production?*
   A: The FastAPI endpoint and ONNX runtime are active, but the Prometheus collector daemon and Grafana dashboard nodes are target architecture currently being set up.
3. **Q:** *Why does Prometheus pull rather than the service pushing metrics?*
   A: Pull-based monitoring prevents the application from hanging if the metrics server goes down or becomes slow, ensuring zero impact on the critical inference path.



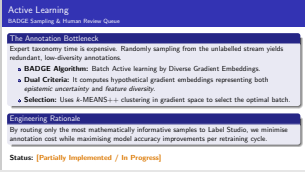
1. **Script:** Let's be fully transparent about our drift monitoring status. We have implemented a 1D Wasserstein scalar distance function in drift.py as an offline prototype, but it is not yet hooked into our live inference pipeline. MMD is our target algorithm for periodic high-dimensional batch audits, but it is not yet implemented in code. This represents high-priority work for our upcoming engineering sprints.
2. **Q:** *Are Wasserstein and MMD currently running in production?*
   A: No. Wasserstein-1 is implemented as an offline evaluation function in drift.py, while MMD and live stream drift alerts are target roadmap specifications.
3. **Q:** *What remains to be done for drift monitoring?*
   A: We need to implement the MMD kernel matrix math, connect Wasserstein drift to live prediction batches, and configure alerting thresholds.



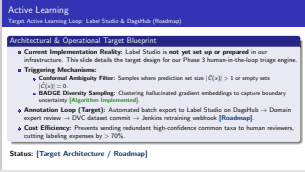
1. **Script:** This slide lists our target metric inventory. Currently in telemetry.py, only the prediction counter is wired up to FastAPI routes. Epistemic metrics like conformal set size and OOD free energy gauges, as well as human referral queue tracking, are defined as constants or design targets awaiting end-to-end integration.
2. **Q:** *Which metrics are actively collected today?*
   A: Only prediction counts grouped by status in predict.py. Conformal set size, latency histograms, and referral rate metrics are defined as constants but await end-to-end wiring.
3. **Q:** *Why measure OOD energy instead of standard softmax probabilities?*
   A: Softmax is notoriously overconfident, pushing unknown inputs to high probability classes. Helmholtz free energy provides a more reliable, unnormalized measure of how far an input is from the training manifold.



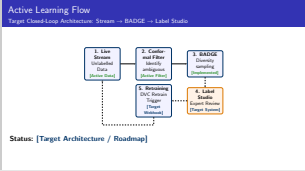
1. **Script:** Machine learning models degrade over time because the real world changes—we call this data drift. We want to be completely open with our teammates about where this system stands: Evidently AI is not yet installed or set up, MMD is not yet coded, and automated drift-triggered retraining is an architectural design on our Phase 3 roadmap. There is significant, impactful engineering work left to build here.
2. **Q:** *Is Evidently AI currently operational in TaxonVision?*
   A: No. Evidently AI is our planned tool for drift reports and visual dashboards, but it has not been configured or integrated into our repository yet.
3. **Q:** *Why not run multivariate MMD on every single incoming inference request?*
   A: Computing full-manifold MMD across high-dimensional embeddings has quadratic complexity $O(N^2)$ in sample size. We perform 1D Wasserstein distance on projected features in real time and reserve full MMD for periodic offline batch audits.



1. **Script:** Annotation by expert taxonomists is expensive and slow. If you send them 10,000 photos of the same common dandelion, you waste their time. BADGE solves this by analyzing the gradients that each unlabelled image would induce in the model. It uses k-MEANS++ clustering in gradient space to pick a batch of images that are both highly uncertain AND visually diverse. This maximizes the model's accuracy improvement per dollar spent on human labeling.
2. **Q:** *What is the current status of the active learning pipeline?*
   A: The BADGE mathematical sampling algorithm is fully implemented in `src/taxon_vision/active_learning/badge.py`. However, the human review queue integration and automated retraining triggers are in progress.

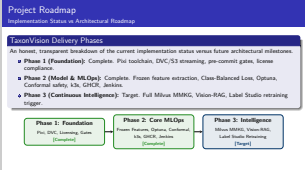


1. **Script:** On this slide, we illustrate our target active learning loop. To be completely clear with the team: Label Studio is not yet set up, configured, or integrated into our deployment. The sampling math in BADGE exists in badge.py, but the web UI, annotation ingestion pipelines, and automated DVC retraining webhooks are target roadmap items that our team will build next.
2. **Q:** *Is Label Studio currently deployed and operational?*
   A: No. There is nothing prepared for Label Studio yet in our infrastructure; it is purely a target architectural blueprint for Phase 3.
3. **Q:** *Why show this workflow if Label Studio is not yet set up?*
   A: To define the exact data contract and trigger mechanisms so that the engineering team can build the Label Studio connector against a unified specification.

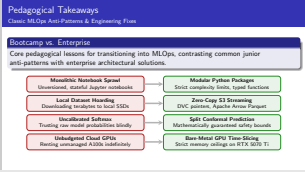


1. **Script:** This flow shows our target architecture for active learning in production. While streaming observations and conformal filtering are active, and BADGE sampling math is implemented in badge.py, automated Label Studio dispatch and closed-loop retraining webhooks remain roadmap goals.
2. **Q:** *Which parts of this 5-step loop are implemented today?*
   A: Steps 1 and 2 (streaming data and conformal filtering) and step 3 math (BADGE) are ready. Steps 4 and 5 (Label Studio annotation and automated retraining webhook) are roadmap items.
3. **Q:** *Why sequence Conformal Filtering before BADGE?*
   A: Conformal filtering quickly isolates ambiguous predictions, shrinking the candidate pool so that BADGE gradient clustering runs in seconds.

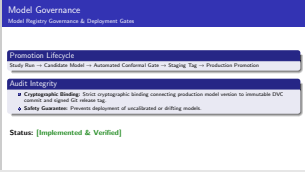
1. **Script:** In Chapter 13, we review our delivery roadmap, contrast common junior anti-patterns with enterprise solutions, and summarize the core engineering discipline required for Level 4 MLOps.
2. **Q:** *What is the core pedagogical takeaway from this chapter?*
   A: Moving beyond stateful exploratory scripts to immutable datasets, mathematical calibration, and hardware-conscious production systems.



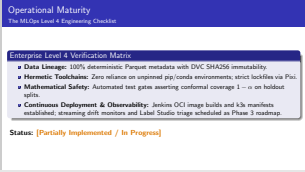
1. **Script:** Let's review our progress transparently. Phase 1 and Phase 2 are complete and battle-tested in our repository today: our reproducible Pixi environment, S3 zero-copy streaming, Class-Balanced Loss runner, conformal safety bounds, Jenkins container builds, and Kubernetes manifests are live. Phase 3—connecting the Milvus vector database into our active learning webhook loop—is our ongoing target architecture roadmap.
2. **Q:** *How do we ensure that new feature development does not regress existing Phase 1 and 2 capabilities?*
   A: Every commit is protected by our pre-commit gates, automated test fixtures in `tests/`, and CI pipeline validation including license and complexity audits.



1. **Script:** For all of you in this course, if you take away four key lessons from this project, make it these four: First, stop writing monolithic notebooks—use modular packages with strict complexity limits. Second, don't download multi-terabyte datasets locally—stream them and cache embeddings. Third, never trust raw softmax probabilities—use conformal prediction sets for safety. And fourth, don't waste thousands of dollars on unmanaged cloud GPUs—budget your VRAM and use time-slicing.
2. **Q:** *What is the fastest way for a junior data scientist to adopt these best practices?*
   A: Start by enforcing automated pre-commit hooks with Ruff and Mypy, tracking datasets with DVC, and writing unit tests for your data transformation functions before touching model code.



1. **Script:** A model registry is not merely a storage bucket; it is a governance boundary. Our promotion lifecycle moves candidates from study runs to staging only after they pass an automated conformal gate. This ensures mathematical safety before human review. For audit integrity, the final production model is cryptographically tied to an immutable DVC commit and a signed Git release tag. This prevents the nightmare scenario of deploying uncalibrated models that have silently drifted from their original training distributions.
2. **Q:** *How does the automated conformal gate prevent bad deployments?*
   A: The gate strictly tests the model on a holdout calibration set. If the empirical coverage falls below $1 - \alpha$ or the set size exceeds a usability threshold, the promotion script halts and rejects the candidate automatically.
3. **Q:** *Why do we require a signed Git release tag for the final promotion?*
   A: The signed tag acts as an immutable, non-repudiable ledger of the human approval step. It guarantees that the deployment authorization was explicitly granted by a recognized principal, fulfilling our audit and compliance requirements.



1. **Script:** This checklist provides our concrete evaluation criteria for Level 4 MLOps maturity. While core data lineage, hermetic toolchains, and mathematical conformal safety gates are fully operational, our end-to-end telemetry daemons and active learning loops remain actively under development.
2. **Q:** *What is the single most critical gate currently enforced before deployment?*
   A: The combination of hermetic Pixi package lockfiles and mathematical conformal coverage verification on holdout data.

TaxonVision-MLOps Architecture

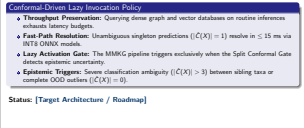
CHAPTER 14

Multimodal Knowledge Graph & Vision-RAG

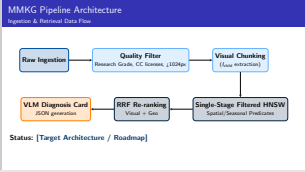
Epistemic Ambiguity Resolution, Filtered Vector Retrieval & Clinical-Grade Diagnosis Cards

Status: [Target Architecture / Roadmap]

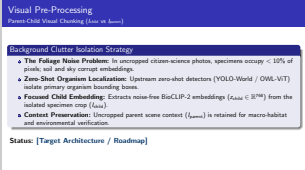
1. **Script:** Welcome to Chapter 14: our capstone architecture on Multimodal Knowledge Graphs and Vision-RAG. Here, we resolve difficult morphological ambiguities through visual chunking, geo-temporal filtered HNSW retrieval, and local VLM differential diagnosis cards.
2. **Q:** *When is the Vision-RAG pipeline triggered?*
   A: It executes lazily only when the upstream conformal predictor detects epistemic ambiguity ($|C| > 1$) or out-of-distribution outliers ($|C| = 0$), preserving fast-path latency for simple classifications.



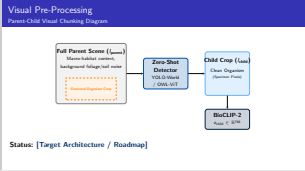
1. **Script:** Our Multimodal Knowledge Graph is powerful, but querying it takes time. In production, we do not query the graph for every single simple observation. If our conformal safety gate outputs a confident singleton prediction ($|\hat{C}(X)| = 1$), we return the INT8 ONNX prediction on the Fast Hot-Path in under 22 milliseconds. We lazy-load the MMKG only when the conformal gate signals uncertainty—either severe ambiguity ($|\hat{C}(X)| > 3$), or an empty prediction set ($|\hat{C}(X)| = 0$) indicating an anomalous, out-of-distribution specimen.
2. **Q:** *Why not query the knowledge graph in parallel on every request?*
   A: Parallel querying would saturate database connection pools and waste CPU/GPU cycles on obvious, clear-cut classifications, degrading overall cluster throughput.



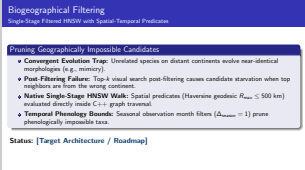
1. **Script:** This diagram maps the full data flow of the MMKG pipeline. Raw data flows through strict quality filters ensuring research-grade, open-license images over 1024 pixels. Then we extract the child organism. The isolated vector is searched in our single-stage filtered HNSW index. Retrieved candidates are fused using Reciprocal Rank Fusion, and finally passed to our local VLM to emit a structured diagnosis card.



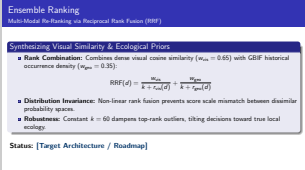
1. **Script:** In nature photography, the actual insect or wildflower often takes up less than 10 percent of the frame. The rest is dirt, grass, or human fingers. If you embed the whole image, your vector captures dirt rather than biology. We use YOLO-World or OWL-ViT to detect the specimen and isolate a clean child bounding box crop for fine-grained BioCLIP-2 embedding, while preserving the parent scene image to verify environmental habitat.
2. **Q:** *What happens if the zero-shot detector finds multiple organisms in one image?*
   A: Each detected child entity generates an independent bounding box and child embedding, allowing multi-organism observations to be analysed separately.



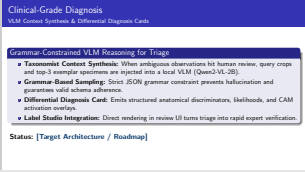
1. **Script:** This diagram visually summarises the Parent-Child Visual Chunking process. We take the full parent scene containing background noise, and pass it through a zero-shot detector like YOLO-World or OWL-ViT. This extracts the child bounding box crop, separating the organism from environmental noise, which is then fed to BioCLIP-2 to compute our clean morphological embedding vector of size 768.



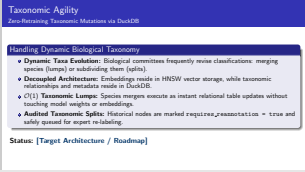
1. **Script:** Nature is full of convergent evolution: two insects on opposite sides of the globe may look identical even though they diverged 50 million years ago. If you only look at pixels, you will misclassify a European wasp as an American cicada killer. In our HNSW vector graph, we evaluate GPS coordinates and seasonal observation month directly inside the graph traversal. If an exemplar is outside a 500-kilometre radius or wrong season, it is pruned instantly before similarity scores are computed.
2. **Q:** *Why not perform vector search first and filter out GPS locations afterward?*
   A: Post-filtering suffers from candidate starvation: if your top-50 visual neighbours are all from the wrong continent, post-filtering discards all of them and leaves you with zero valid results. Native single-stage filtering guarantees high recall among geographically valid candidates.



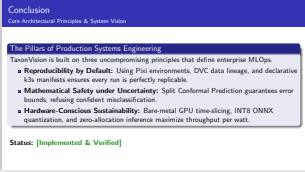
1. **Script:** Once we retrieve candidate species, how do we combine visual similarity with ecological reality? We use Reciprocal Rank Fusion. We assign a 65 percent weight to visual similarity and a 35 percent weight to historical GBIF occurrence density. Even if an exotic species looks slightly closer visually, a high local occurrence density for the native sibling species correctly tilts the ranking towards the true ecological observation.
2. **Q:** *What is the parameter k in the RRF formula?*
   A: The constant k (typically set to 60) dampens the impact of extreme outliers at top ranks, preventing a single high rank from disproportionately dominating the final ensemble score.



1. **Script:** When a human taxonomist is asked to review an ambiguous specimen, we don't just dump raw images on their screen. We pass the query crop and the top-3 reference exemplars into a local Vision-Language Model like Qwen2-VL-2B. Constrained by grammar-based decoding, the VLM generates a structured JSON differential diagnosis card: it details specific anatomical discriminators, shows CAM overlays, and allows the expert to confirm the species in seconds.
2. **Q:** *Why use grammar-based sampling with the Vision-Language Model?*
   A: Unconstrained VLMs hallucinate text and output unpredictable schemas. Grammar-based sampling forces the model to emit strictly valid JSON conforming to our differential diagnosis schema.



1. **Script:** In real biology, taxonomy is not static. Scientists frequently merge two species into one, or split one species into three. In a standard deep learning model, every split or merge requires retraining the entire neural network from scratch. Because our architecture decouples HNSW vector storage from DuckDB relational metadata, a taxonomic merge is an instant O(1) database update. Our vectors remain untouched with zero GPU compute cost.
2. **Q:** *What happens during a taxonomic split when one species becomes two?*
   A: Historical observation nodes for that taxon in DuckDB are flagged with `requires_reannotation = true` and purged from active traversal. Their crops are routed to Label Studio for expert re-sorting, after which they are re-registered under their new respective taxon keys.



1. **Script:** To conclude our complete presentation: MLOps is not just about using trendy tools like Kubernetes or Docker. It is a systems engineering mindset. It means building models that are reproducible down to the exact commit, mathematically honest about what they don't know, and engineered to run reliably on physical hardware within budget. TaxonVision proves that with the right architecture, we can tackle the most complex biodiversity challenges on Earth with mathematical confidence and operational excellence. Thank you!
2. **Q:** *Where can students inspect the source code and documentation?*
   A: The entire repository, including Zensical documentation, LaTeX strategy reports, and CI/CD pipelines, is maintained under strict version control in the taxon-vision-mlops repository.